

Embedded
Online
Conference

The Power of a Look-up Table

www.embeddedonlineconference.com

Nathan Jones

Sine LUT

```
                // sin(0*PI/180)          sin(1*PI/180) ...          sin(359*PI/180)
const double sin_LUT[360] = {          0, 0.0174524064372835, ... -0.0174524064372845 };

double sin_4 = sin_LUT[4]           // Equivalent to sin(4*PI/180)
double sin_354 = sin_LUT[354]       // Equivalent to sin(354*PI/180)
```

What can a LUT do for you?

1. Make portions of your program faster
2. Clean up your conditionals (make your program easier to read/change)
3. Blur the line between HW and SW
4. Give you simple run-time configurability/polymorphism
5. Capture your program's knowledge as data, not hard-coded logic



The single most powerful software technique that I use to do impossible things is LUTs.

- Charles Lohr, “Accomplishing the Impossible with LUTs” (paraphrased)

AGENDA

1

Intro and Usefulness

2

Fast math

3

Cleaning up conditionals

4

Simple Polymorphism /
Run-time configurability

5

Capturing program
knowledge as data

6

Summary



Fast Math

Sine LUT

```
                // sin(0*PI/180)          sin(1*PI/180) ...          sin(359*PI/180)
const double sin_LUT[360] = {          0, 0.0174524064372835, ... -0.0174524064372845 };

double sin_4 = sin_LUT[4]           // Equivalent to sin(4*PI/180)
double sin_354 = sin_LUT[354]       // Equivalent to sin(354*PI/180)
```

Sine LUT

```

// sin(0*PI/180)      sin(1*PI/180) ...      sin(359*PI/180)
const double sin_LUT[360] = {      0, 0.0174524064372835, ... -0.0174524064372845 };

double sin_4 = sin_LUT[4]      // Equivalent to sin(4*PI/180)
double sin_354 = sin_LUT[354] // Equivalent to sin(354*PI/180)
```

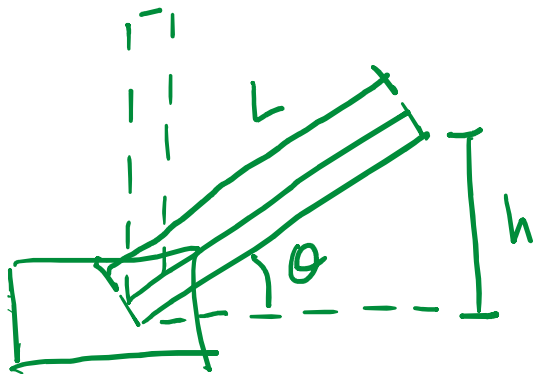
	Memory usage (bytes)	Max absolute error	Percent error	Execution time	Speed-up factor
Library sin	5480	N/A	N/A	~47 us	N/A
LUT sin	3484	~0.0076	~2.5	~6.7 us	7.04x

Sine LUT

```
// sin(0*PI/180)          sin(1*PI/180) ...          sin(359*PI/180)
const double sin_LUT[360] = {      0, 0.0174524064372835, ... -0.0174524064372845 };

double sin_4 = sin_LUT[4]          // Equivalent to sin(4*PI/180)
double sin_354 = sin_LUT[354]     // Equivalent to sin(354*PI/180)
```

	Memory usage (bytes)	Max absolute error	Percent error	Execution time	Speed-up factor
Library sin	5480	N/A	N/A	~47 us	N/A
LUT sin	3484	~0.0076	~2.5	~6.7 us	7.04x



$h = L \sin \theta$
Max error (h) = 0.0076L,
or ~0.23 mm for L = 30 cm

Sine LUT

```
// sin_LUT.h
double sin_LUT(double rad);

// sin_LUT.c
double sin_LUT(double rad) {
    // 2pi*64 = 402.12...
    static const double sin_LUT[404] = {...};

    int idx = (int)(rad*64+0.5)
    return sin_LUT[idx];
}

// main.c
#include "sin_LUT.h"
double sin_1rad = sin_LUT(1);
double sin_0pt34rad = sin_LUT(0.34);
```

Sine LUT

```
// sin_LUT.h  
double sin_LUT(double rad);
```

- Scope: Function-local
- Storage location: Flash, not RAM (hopefully)

```
// sin_LUT.c  
double sin_LUT(double rad) {  
    // 2pi*64 = 402.12...  
    static const double sin_LUT[404] = {...};
```

- Elements: Doubles
- Dimensions: 1
- Table computed: Manually

```
    int idx = (int)(rad*64+0.5);  
    return sin_LUT[idx];  
}
```

- Addressing method: Hashed
 - Don't forget to add bounds-checking!!

```
// main.c  
#include "sin_LUT.h"  
double sin_1rad = sin_LUT(1);  
double sin_0pt34rad = sin_LUT(0.34);
```

Other improvements and alternate solutions

Other improvements

- Piece-wise linear interpolation (PwLI)
- Sparse LUT
- Returning floats, fixed-point values

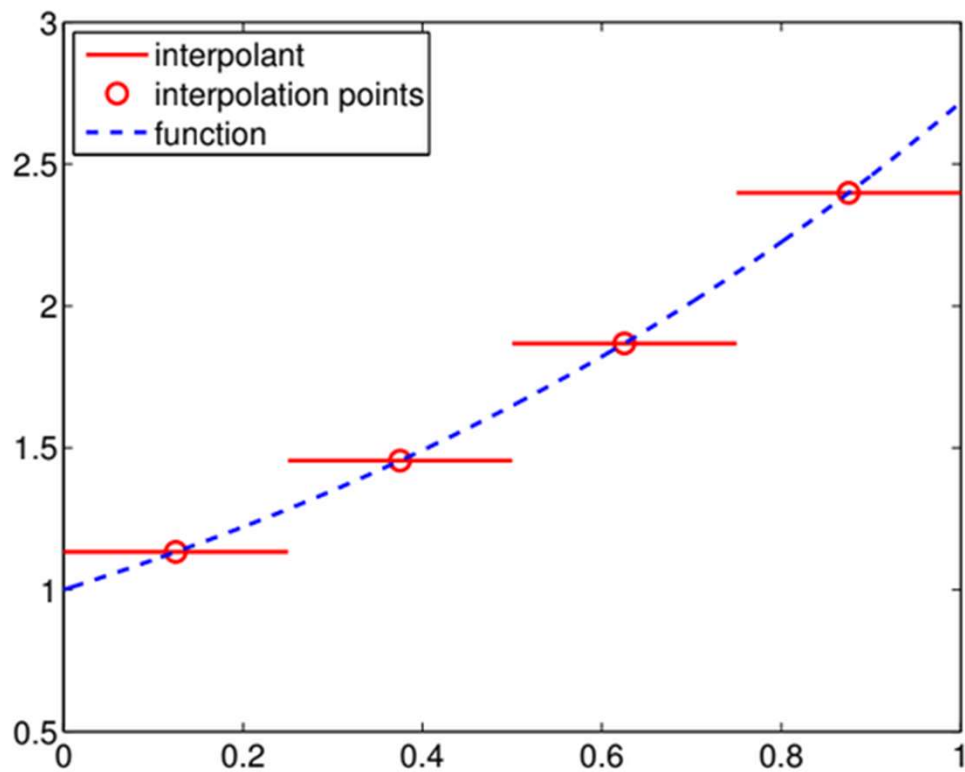
Alternate solutions

- Polynomial approximation
- Memoization

Other improvements and alternate solutions

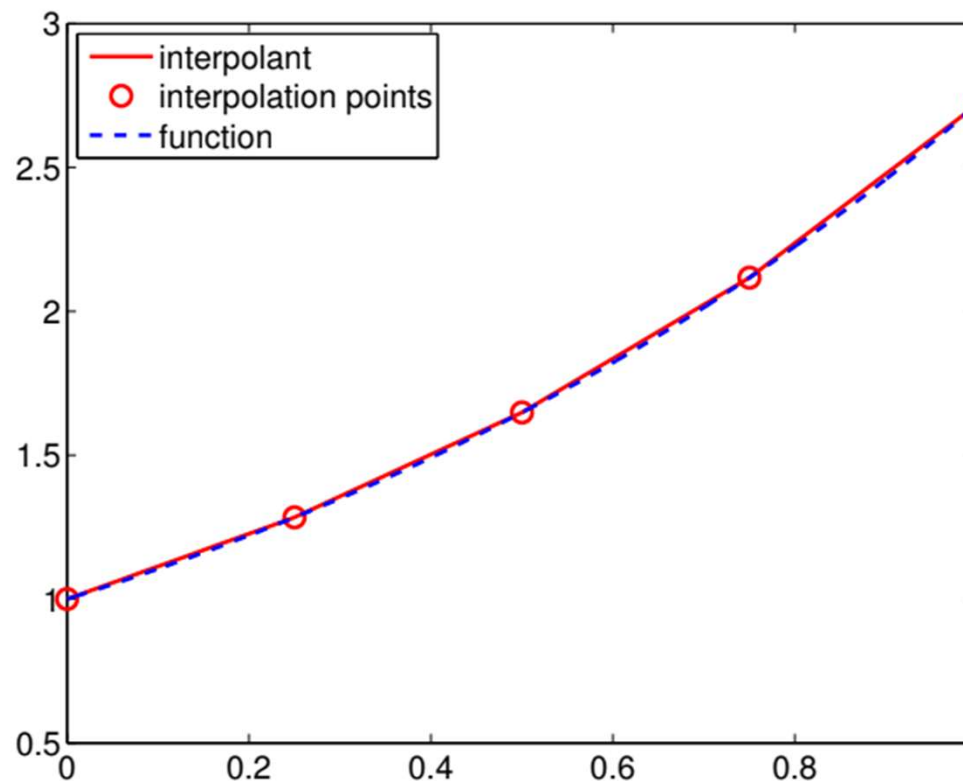
Other improvements

- **Piece-wise linear interpolation (PwLI)**
- Sparse LUT
- Returning floats, fixed-point values



Alternate solutions

- Polynomial approximation
- Memoization



Source: tinyurl.com/38kzm97j

Other improvements and alternate solutions

Other improvements

- Piece-wise linear interpolation (PwLI)
- Sparse LUT
- Returning floats, fixed-point values

Alternate solutions

- Polynomial approximation
- Memoization


MICROCHIP

AN942

Piecewise Linear Interpolation on PIC12/14/16 Series Microcontrollers

Author: John Day and Steven Bible
Microchip Technology Inc.

INTRODUCTION

The embedded systems world is filled with sensing and measurement techniques that present non-linear output results. If the sensing technique is well characterized, a mathematical transfer function can be used to correct and linearize the sensor output. In many cases, these functions contain complex polynomial and exponential functions, placing a great burden on the program memory, RAM and execution speed of most low-cost microcontrollers.

This application note will explore a simple Piecewise Linear Interpolation technique that is often employed to linearize a sensor's output. This technique has many advantages in faster execution speed using a simple look-up table and significantly reduced program memory and RAM requirements. However, this technique can introduce a considerable error compo-

EQUATION 1: STEINHART-HART

$$\frac{1}{T_K} = A + B(\ln R) + (C \ln R)^3$$

Solving for temperature in degrees Celsius:

$$T_{\text{Degrees C}} = \frac{1}{A + B(\ln R) + C(\ln R)^3} - 273.15$$

Legend:
A, B, C = Constants determined through a calibration process
R = The thermistor's resistance in ohms
T_K = The temperature in Kelvins

FIGURE 1: TEMPERATURE VS. RESISTANCE

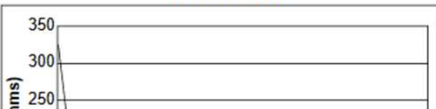
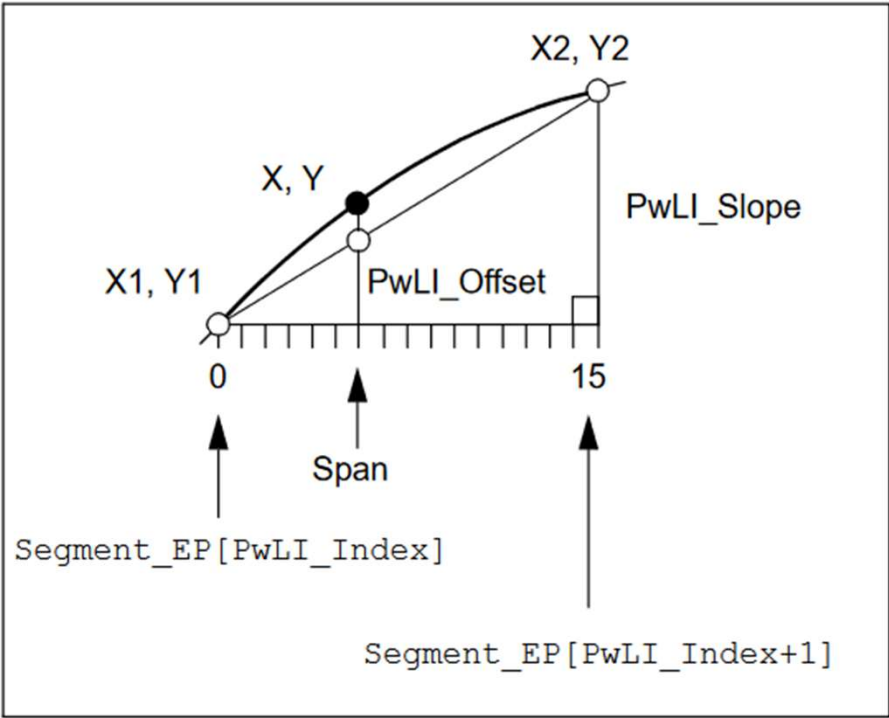


FIGURE 5: CALCULATING INTERMEDIATE POINT WITHIN THE SEGMENT



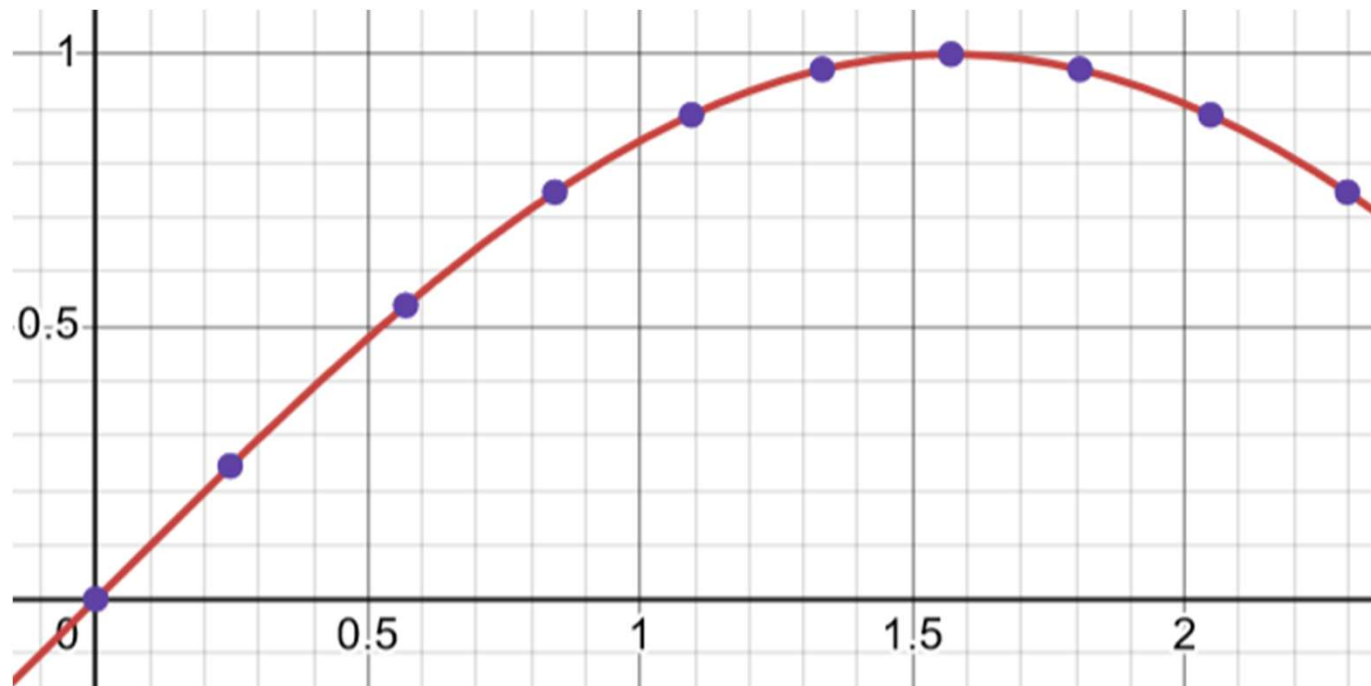
Other improvements and alternate solutions

Other improvements

- Piece-wise linear interpolation (PwLI)
- **Sparse LUT**
- Returning floats, fixed-point values

Alternate solutions

- Polynomial approximation
- Memoization



Other improvements and alternate solutions

Other improvements

- Piece-wise linear interpolation (PwLI)
- **Sparse LUT**
- Returning floats, fixed-point values

Alternate solutions

- Polynomial approximation
- Memoization

```
static point_float_t nonUniform_float_0dot007error[NONUNIFORM_0DOT007ERROR_SIZE] =
{
    { 0.00000000000000000000, 0.00000000000000000000 },
    { 0.246907827863357000, 0.244406737256656000 },
    { 0.569239513635344000, 0.538991638538422000 },
    { 0.843093840424931000, 0.746704576346488000 },
    ...
}
```


Other improvements and alternate solutions

Other improvements

- Piece-wise linear interpolation (PwLI)
- Sparse LUT
- **Returning floats, fixed-point values**

Double

- $\pm 2.23 \times 10^{-308}$ to $\pm 1.80 \times 10^{308}$

Float

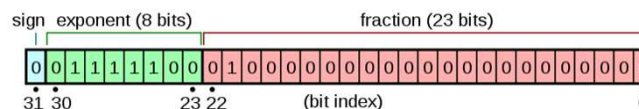
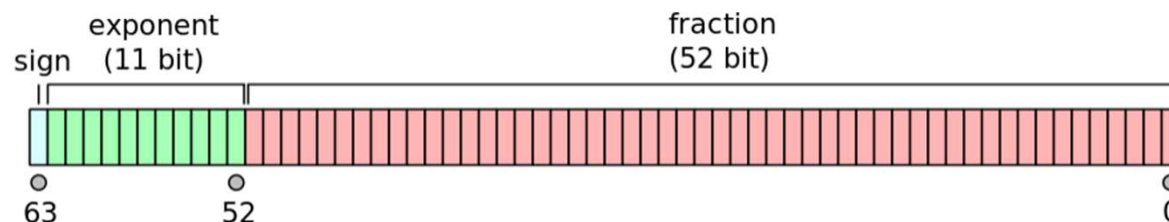
- $\pm 1.18 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$

Fixed-point

- Q7.8: [-128, 127.99609375] in increments of 0.00390625
- Q0.31: [-1, 0.999999999976] in increments of $2.3283064 \times 10^{-10}$

Alternate solutions

- Polynomial approximation
- Memoization



Binary Representation

1	1	0	1	0	1	1	0	1	1	0	0	1	1	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Q7.8 Fixed Point Representation

1	1	0	1	0	1	1	0	1	1	0	0	1	1	0	1
-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}

-42 • 80078125

Sources: tinyurl.com/5e9kp38x
[3v6jyae8](https://tinyurl.com/3v6jyae8)
[28754d5a](https://tinyurl.com/28754d5a)

Other improvements and alternate solutions

Other improvements

- Piece-wise linear interpolation (PwLI)
- Sparse LUT
- Returning floats, fixed-point values

Alternate solutions

- **Polynomial approximation**
- Memoization

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots,$$

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots,$$

$$\exp(x) = \frac{x^0}{0!} + \frac{x^1}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots,$$

$$\log(x+1) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \frac{x^5}{5} + \dots,$$

<-- Taylor series, also

- Chebyshev polynomial
- Bessel functions

Source: tinyurl.com/y57p8azm

Other improvements and alternate solutions

Other improvements

- Piece-wise linear interpolation (PwLI)
- Sparse LUT
- Returning floats, fixed-point values

Alternate solutions

- Polynomial approximation
- **Memoization**

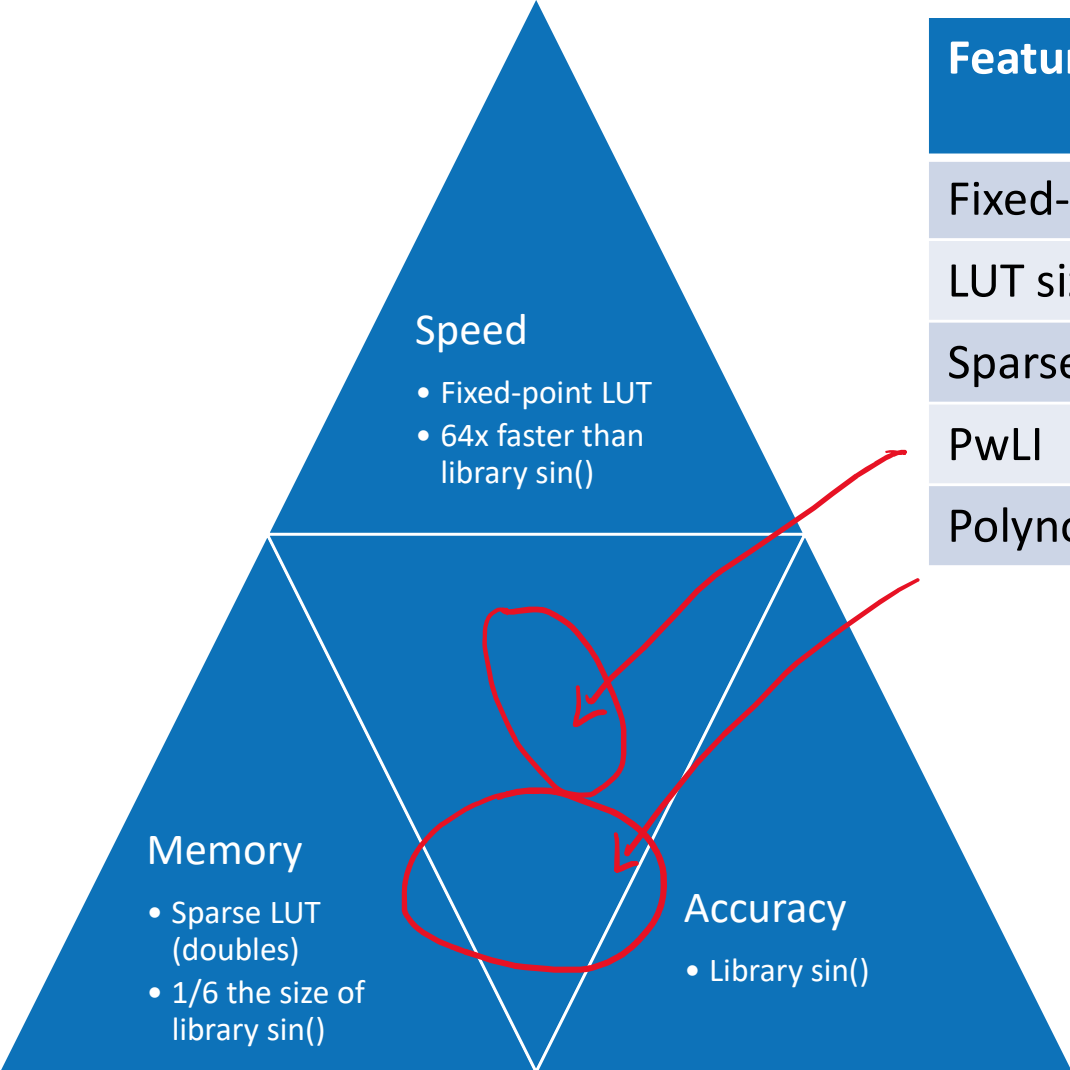
```
FUNCTION sin_lut( rad )
```

```
    previous_values = { {in, out}, {in, out}, {in, out}, ... }
```

```
    IF rad IN previous_values:  
        return previous_values[rad]
```

```
    ELSE:  
        Normal look-up or function call
```

Comparison of fast sin solutions



Feature	Use more/Move to the right for better...	...and worse...
Fixed-point / Float / Double	Accuracy	Memory / Speed
LUT size	Accuracy	Memory
Sparse LUT	Memory	Accuracy / Speed
PwLI	Accuracy	Speed
Polynomial approx.	Accuracy	Memory / Speed

(PwLI: Piece-wise Linear Interpolation)

Other uses for fast math/simple look-up

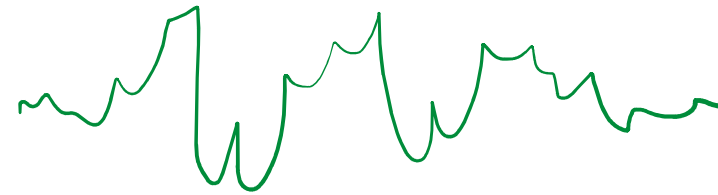
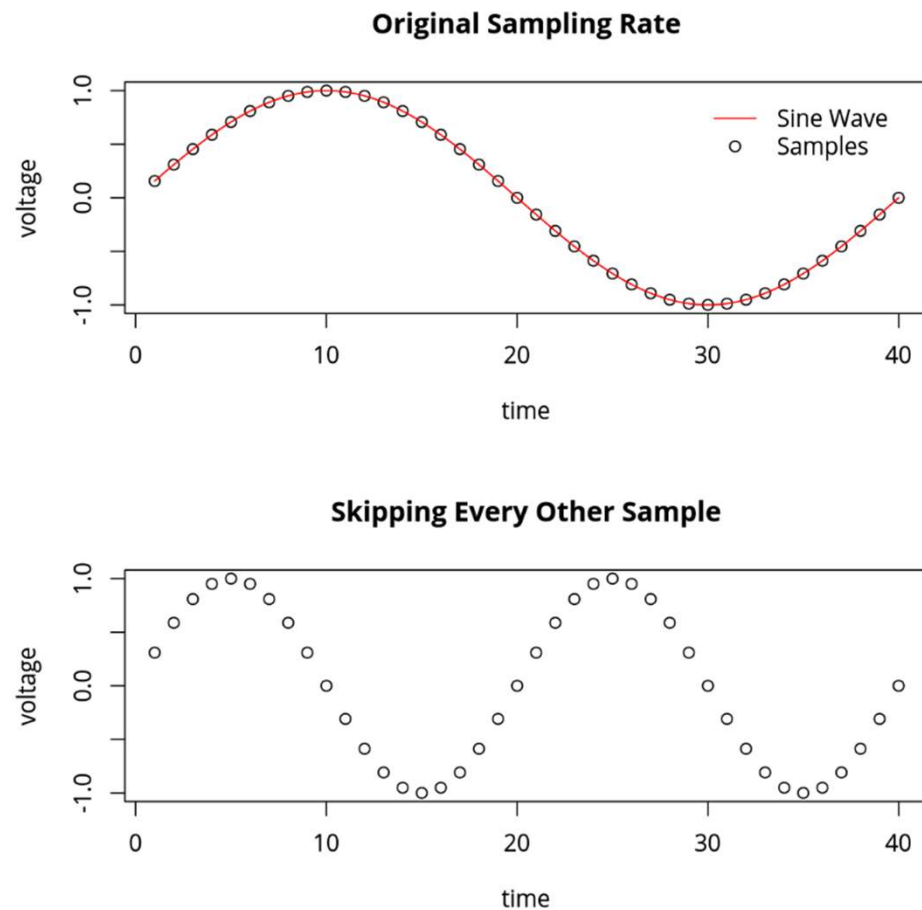
- Cyclic redundancy checks (CRC)¹
- Portions of encryption/decryption (can recompute at run-time when key changes)
- Bit-twiddling (number of ones, parity, reversing bit sequences, etc)²
 - “Super Simple Tasker” by Miro Samek uses LUT to determine highest priority ready-to-run task³
- Converting ADC values to physical values
- Bitmaps
- Direct digital synthesis (DDS)
- Channel 3 TV, Zlib decompression (Charles Lohr)

1: tinyurl.com/2uw8k77n

2: tinyurl.com/54ukckyb

3: tinyurl.com/yezdfyae

Direct Digital Synthesis (DDS)

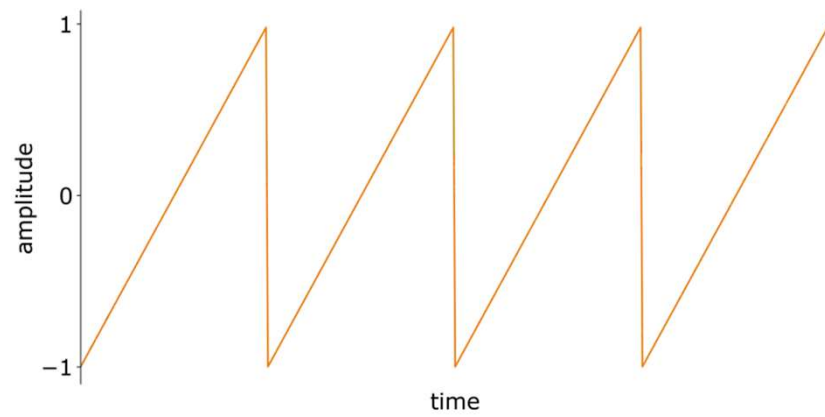


Source: tinyurl.com/56s85cbj

Figure 13-1. Sampled sine wave

Direct Digital Synthesis (DDS)

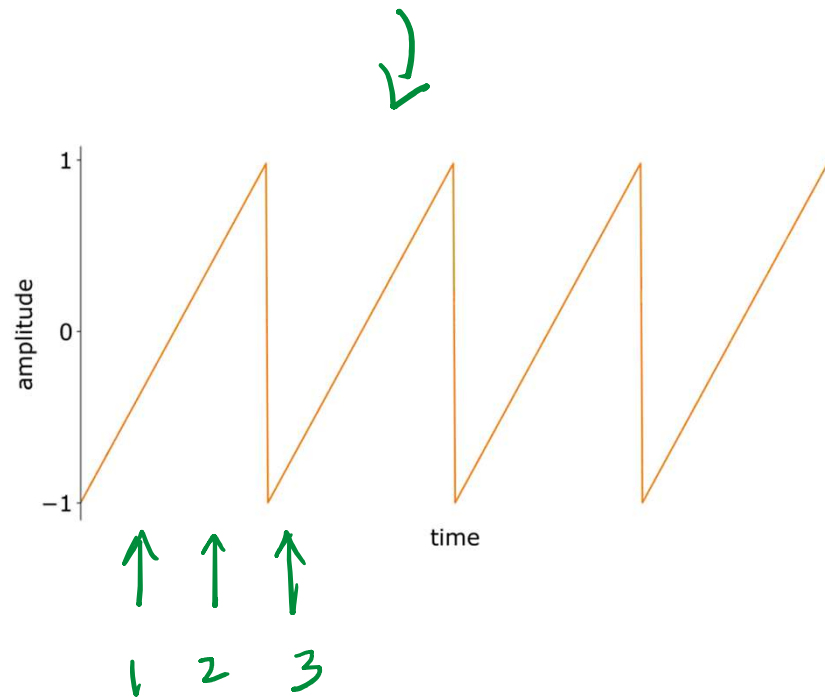
```
double sawtooth[] = { ... };
```



Source: tinyurl.com/56s85cbj

Direct Digital Synthesis (DDS)

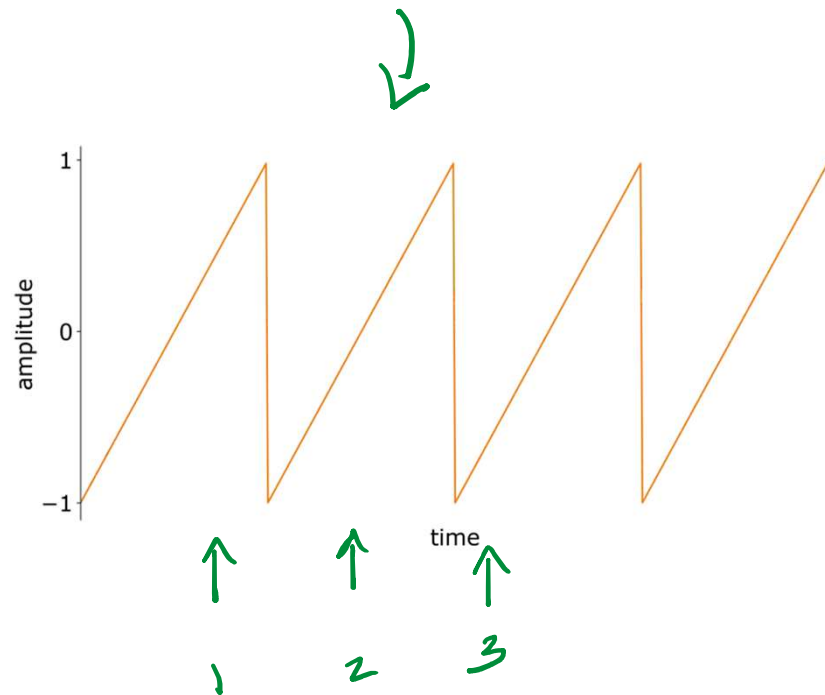
```
double sawtooth[] = { ... };
```



Source: tinyurl.com/56s85cbj

Direct Digital Synthesis (DDS)

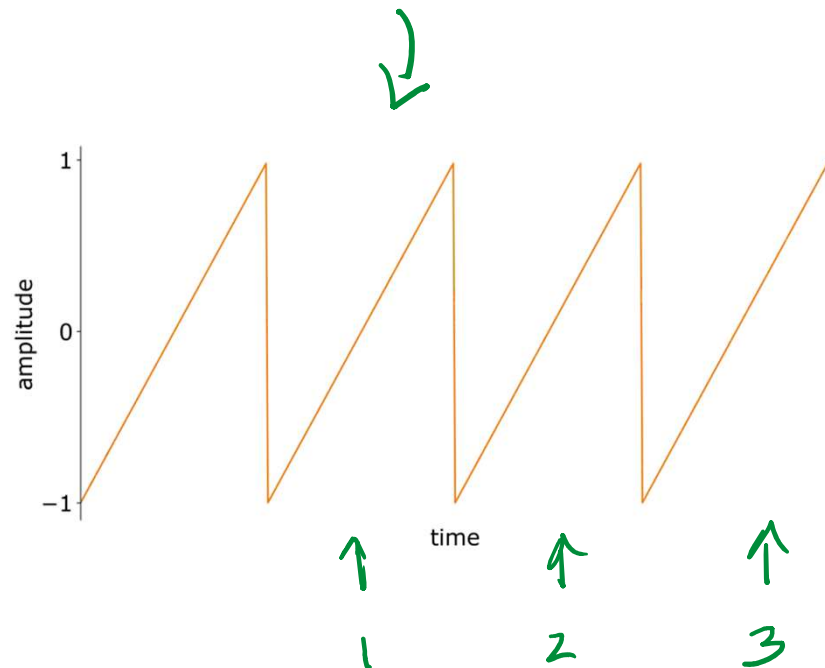
```
double sawtooth[] = { ... };
```



Source: tinyurl.com/56s85cbj

Direct Digital Synthesis (DDS)

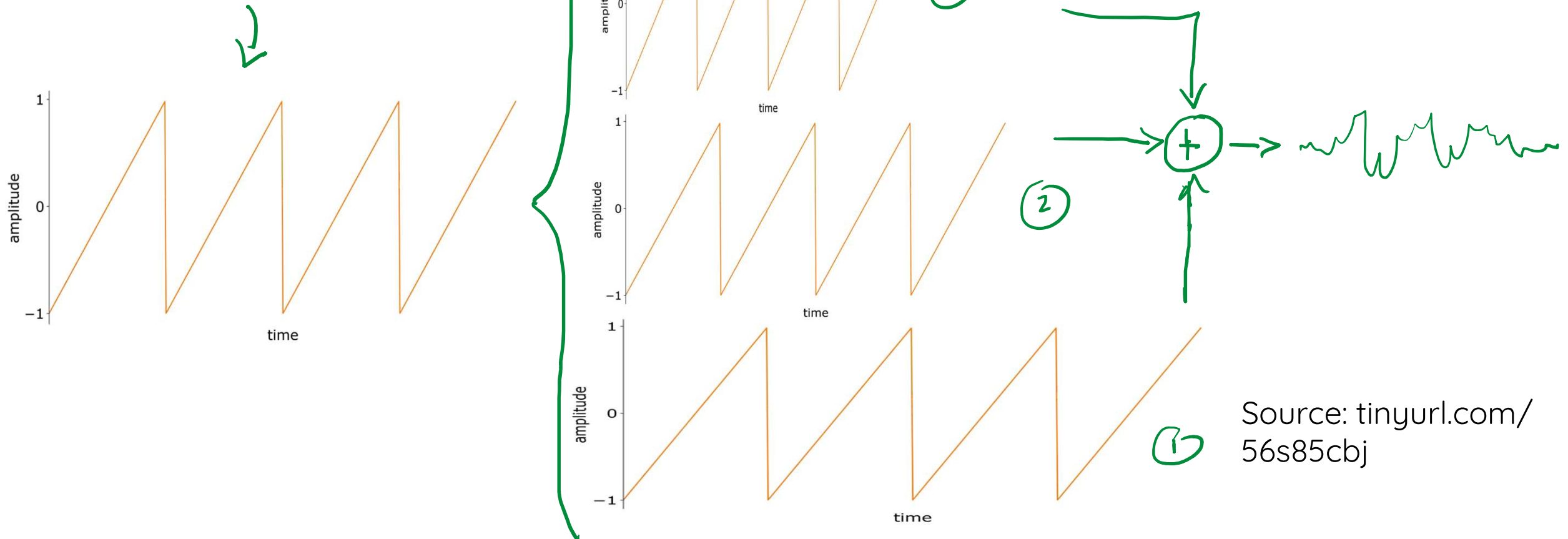
```
double sawtooth[] = { ... };
```



Source: tinyurl.com/56s85cbj

Direct Digital Synthesis (DDS)

```
double sawtooth[] = { ... };
```



Challenge Question(s)

- What are the benefits of encapsulating LUT access in a function? For example:

```
double sin_LUT(double rad) {  
    // 2pi*64 = 402.12...  
    static const double sin_LUT[404] = {...};  
  
    int idx = (int)(rad*64+0.5)  
    return sin_LUT[idx];  
}
```

- What are the advantages and disadvantages of having a LUT in Flash vs Ram?
- Think of a recent project you were working on. What part of that code could have benefitted from the “fast math” provided by a LUT?

Challenge Question(s)

- What are the benefits of encapsulating LUT access in a function?
 - Encapsulation of the LUT
 - Can group hash function or PwLI together with the LUT
- What are the advantages and disadvantages of having a LUT in Flash vs Ram?
 - Flash: Harder to modify LUT at run-time, but usually more space than RAM
 - RAM: Sometimes faster to access than Flash, More easy to modify at run-time, Usually less space than Flash
- Think of a recent project you were working on. What part of that code could have benefitted from the “fast math” provided by a LUT?



Cleaning up Conditionals

Insurance rates

Conditionals, v1

```
if (gender == FEMALE) {  
    if (smokingStatus == NON_SMOKING) {  
        if (maritalStatus == MARRIED) myInsuranceRate = 100;  
        else myInsuranceRate = 200;  
    }  
    else {  
        if (maritalStatus == MARRIED) myInsuranceRate = 300;  
        else myInsuranceRate = 300;  
    }  
} else {  
    if (smokingStatus == NON_SMOKING) {  
        if (maritalStatus == MARRIED) myInsuranceRate = 400;  
        else myInsuranceRate = 300;  
    }  
    else {  
        if (maritalStatus == MARRIED) myInsuranceRate = 400;  
        else myInsuranceRate = 400;  
    }  
}
```

Insurance rates

Conditionals, v2

```
bool x = (gender == MALE);  
bool y = (smokingStatus == SMOKING);  
bool z = (maritalStatus == SINGLE);  
  
if ( ( x && !z ) || ( x && y && z ) )      myInsuranceRate = 400;  
else if ( ( !x && y ) || ( x && !y && z ) ) myInsuranceRate = 300;  
else if ( !x && !y && z )                 myInsuranceRate = 200;  
else                                     myInsuranceRate = 100;
```


Insurance rates

LUT

```
typedef enum { FEMALE, MALE, NUM_GENDERS } gender_t;
typedef enum { NON_SMOKING, SMOKING, NUM_SMOKING_STATUSES } smokingStatus_t;
typedef enum { MARRIED, SINGLE, NUM_MARITAL_STATUSES } maritalStatus_t;

int insuranceRates[NUM_GENDERS][NUM_SMOKING_STATUSES][NUM_MARITAL_STATUSES] = {
    // GENDER   SMOKING STATUS   MARITAL STATUS
    // -----
    100, // FEMALE   NON-SMOKING   MARRIED
    200, // FEMALE   NON-SMOKING   SINGLE
    300, // FEMALE   SMOKING     MARRIED
    300, // FEMALE   SMOKING     SINGLE
    400, // MALE     NON-SMOKING   MARRIED
    300, // MALE     NON-SMOKING   SINGLE
    400, // MALE     SMOKING     MARRIED
    400, // MALE     SMOKING     SINGLE
}

int myInsuranceRate = insuranceRates[MALE][NON_SMOKING][MARRIED] // 400
```



Virtually anything you can select with logic statements, you can select with tables instead. In simple cases, logic statements are easier and more direct. As the logic chain becomes more complex, tables become increasingly attractive.

- Steve McConnell, "Code Complete"

Inputs that aren't contiguous positive integers

- $[0,4]$ and $[6,9]$?
- $[-5,5]$?
- $[0.12,0.89]$?
- $[0,60)$ and $[60,70)$ and $[70,80)$ and $[80,90)$ and $[90,100]$?

Inputs that aren't contiguous positive integers

- $[0,4]$ and $[6,9]$?
- $[-5,5]$?
- $[0.12,0.89]$?
- $[0,60)$ and $[60,70)$ and $[70,80)$ and $[80,90)$ and $[90,100]$?

- Only populate the array elements you care about
- Indirect access
- Hash function
- Searching (key/value)

Inputs that aren't contiguous positive integers

- [0,4] and [6,9]?
- [-5,5]?
- [0.12,0.89]?
- [0,60) and [60,70) and [70,80) and [80,90) and [90,100]?

- **Only populate the array elements you care about**

```
assert( input <= 4 || ( input >= 6 && input <= 9 ) );  
int vals[] = { 1, 3, 5, 7, 2, 0, 8, 9, 7, 5 };  
return vals[ input ];
```

- Indirect access
- Hash function
- Searching (key/value)

Inputs that aren't contiguous positive integers

Indirect Access

- First table (accessed by input value) holds the *array indices*, second table holds the actual data values

- Ex:

```
int indices[] = {1, 6, 3, 0, 0, 0, 4, 5, 2}
```

```
int vals[] = {0, 100, 200, 300, 400, 500, 600}
```

```
int value_I_want = vals[ indices[ input ] ]; // Input is either [0,2] or [6,8]
```

- Also allows for multiple ways of accessing a table

```
int productIDs[] = { [1001] = 0, [2304] = 1 };
```

```
product_t database[] = { { /* Product A */ }, { /* Product B */ }, };
```

```
int shelfLocation[] = { [56] = 1, [75] = 0 };
```

```
product_t productWithID2304 = database[ productIDs[ 2304 ] ];
```

```
product_t productAtLocation56 = database[ shelfLocation[ 56 ] ];
```

Inputs that aren't contiguous positive integers

Hashing

- Process of converting an input into an array index
- Hash functions

- Trivial hash: `sin_LUT[4]`

- Actual hash ----->

- Simple expression/Ternary
`sin_LUT[ (int) (rad*64+0.5) ];`
`LUT[ (x < 0) ? 0 : x ];`

- Statement expression (GCC extension)
`LUT[ ({ while( x > 404 ) x--; x; }) ];`

- Actual function
`LUT[ getIndex( x ) ];`

```
static uint64_t hash_key(const char* key) {  
    uint64_t hash = FNV_OFFSET;  
    for (const char* p = key; *p; p++) {  
        hash ^= (uint64_t)(unsigned char)(*p);  
        hash *= FNV_PRIME;  
    }  
    return hash;  
}
```

Source: tinyurl.com/22p5a9cw

Inputs that aren't contiguous positive integers

Searching

- When LUT is organized as key-value pairs (e.g. in a sparse LUT)

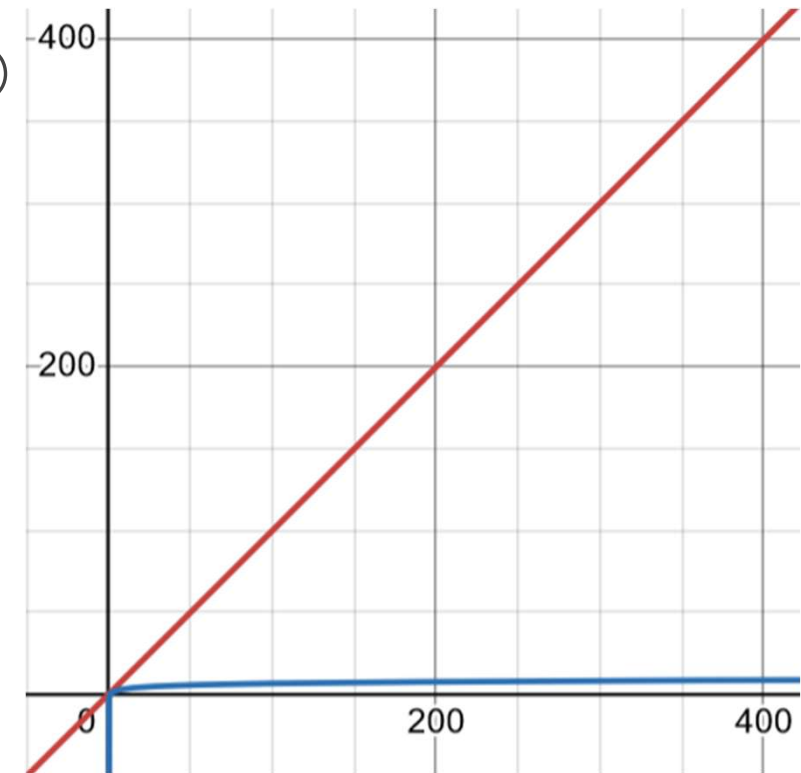
```
static point_float_t arr[] = {  
    { 0.00000000000000000000, 0.00000000000000000000 },  
    { 0.246907827863357000,  0.244406737256656000 }, ... }
```

- Linear search (red) completes in $O(n)$

```
for( int idx = 0; idx < LEN_LUT; idx++ ) {  
    if( lut[ idx ].x_val == input ) break;  
    return lut[ idx ].y_val;  
}
```

- Binary search (blue) completes in $O(\log_2 n)$

```
void *bsearch( const void *key,  
               const void *base,  
               size_t nitems,  
               size_t size,  
               int (*compar)(const void *, const void *) )
```



Challenge Question(s)

- Convert the following conditional into a LUT:

```
typedef enum { UNKNOWN, LETTER, PUNCTUATION, DIGIT } charType_t;
```

```
if( ( ( 'a' <= inputChar ) && ( inputChar <= 'z' ) ) ||  
    ( ( 'A' <= inputChar ) && ( inputChar <= 'Z' ) ) ) {  
    charType = LETTER;  
}  
else if( ( inputChar == ' ' ) || ( inputChar == ',' ) || ( inputChar == '.' ) ||  
        ( inputChar == '!' ) || ( inputChar == '(' ) || ( inputChar == ')' ) ||  
        ( inputChar == ':' ) || ( inputChar == ';' ) || ( inputChar == '?' ) ||  
        ( inputChar == '-' ) ) {  
    charType = PUNCTUATION;  
}  
else if( ( '0' <= inputChar ) && ( inputChar <= '9' ) ) {  
    charType = DIGIT;  
}
```

Challenge Question(s)

- Convert the following conditional into a LUT:

```
typedef enum { UNKNOWN, LETTER, PUNCTUATION, DIGIT } charType_t;
```

```
charType_t charTypes[] = { ['A'] = LETTER, ['B'] = LETTER, ['C'] = LETTER, ...  
                           ['a'] = LETTER, ['b'] = LETTER, ['c'] = LETTER, ...  
                           [' ' ] = PUNCTUATION, ['\',''] = PUNCTUATION, ...  
                           ['0'] = DIGIT,   ['1'] = DIGIT,   ['2'] = DIGIT, };
```

```
charType = charTypes[inputChar];
```

Challenge Question(s)

- Write a LUT for the following code description: Return the status of a car engine (NORMAL, CONCERN, WARNING, or DANGER) based on the engine's mode (STANDARD or TURBO), it's internal temperature (see ranges below), and it's RPMs (see ranges below).

Standard					
	RPMs				
Temps	<5000	[5k,6k)	[6k,7k)	[7k,8k)	>8k
<20.9					
[20.9, 25.9)					
[25.9,29.8)					
[29.8,34.8)					
[34.8,40.0)					
>40.0					
Legend:	Normal	Concern	Warning	Danger	

Turbo					
	RPMs				
Temps	<5000	[5k,6k)	[6k,7k)	[7k,8k)	>8k
<20.9					
[20.9, 25.9)					
[25.9,29.8)					
[29.8,34.8)					
[34.8,40.0)					
>40.0					
Legend:	Normal	Concern	Warning	Danger	

Challenge Question(s)

- Step 1: Make a 2-element LUT based on engine setting

```
typedef enum { NORMAL, CONCERN, WARNING, DANGER } status_t;  
typedef enum { STANDARD, TURBO, NUM_SETTINGS } engineSettings_t;  
  
Status_t engineStatuses[NUM_SETTINGS] = {...};  
  
return engineStatuses[TURBO];
```

Challenge Question(s)

- Step 2: Subdivide that LUT into NUM_TEMPS elements and make an array of floats corresponding to the limits of the temperature ranges. Use a linear search (inside a statement expression) to get the proper index.

```
#define NUM_TEMPS 5
typedef enum { NORMAL, CONCERN, WARNING, DANGER } status_t;
typedef enum { STANDARD, TURBO, NUM_SETTINGS } engineSettings_t;

float temps[NUM_TEMPS] = {20.9, 25.9, 29.8, 34.8, 40};
Status_t engineStatuses[NUM_SETTINGS][NUM_TEMPS+1] = {...};

engineSettings_t engineSetting = TURBO;
float setTemp = 31.7;

return engineStatuses[engineSetting]
    [({int idx = 0; for(; (idx < NUM_TEMPS) && (temps[idx] < setTemp); idx++){}} idx;)];
```

Challenge Question(s)

- Step 3: Subdivide the LUT one more time into MAX_kRPMs elements, one for each integer value of RPMs (e.g. 0, 1, 2, ... 9); elements at indices 0-4 are redundant (all are <5000 RPMs), as are the elements at indices 8 and 9 (both are >8000 RPMs). Find RPM index by dividing by 1000 and converting to int.

```
#define NUM_TEMPS 5
#define MAX_kRPMs 10

typedef enum { NORMAL, CONCERN, WARNING, DANGER } status_t;
typedef enum { STANDARD, TURBO, NUM_SETTINGS } engineSettings_t;

float temps[NUM_TEMPS] = {20.9, 25.9, 29.8, 34.8, 40};
Status_t engineStatuses[NUM_SETTINGS][NUM_TEMPS+1][MAX_kRPMs] = {...};

engineSettings_t engineSetting = TURBO;
float setTemp = 31.7;
int RPMs = 6043;

return engineStatuses[engineSetting]
    [({int idx = 0; for(; (idx < NUM_TEMPS) && (temps[idx] < setTemp); idx++){}} idx;))]
    [(int)(RPMs/1000)];
```

Challenge Question(s)

- Step 4: Complete table of values.

```
status_t engineStatuses[NUM_SETTINGS][NUM_TEMPS+1][MAX_kRPMs] = {
```

```
// Engine setting: Normal
// RPMs:
//0      1000      2000      3000      4000      [5k,6k) [6k,7k) [7k,8k) 8000      9000
NORMAL,  NORMAL,  NORMAL,  NORMAL,  NORMAL,  CONCERN, CONCERN, CONCERN, WARNING, WARNING, // <20.9 Temps
CONCERN, CONCERN, CONCERN, CONCERN, CONCERN, CONCERN, CONCERN, WARNING, WARNING, WARNING, // [20.9,25.9)
CONCERN, CONCERN, CONCERN, CONCERN, CONCERN, WARNING, WARNING, WARNING, WARNING, WARNING, // [25.9,29.8)
CONCERN, CONCERN, CONCERN, CONCERN, CONCERN, WARNING, WARNING, DANGER, DANGER, DANGER, // [29.8,34.8)
WARNING, WARNING, WARNING, WARNING, WARNING, WARNING, DANGER, DANGER, DANGER, DANGER, // [34.8,40.0)
WARNING, WARNING, WARNING, WARNING, WARNING, WARNING, DANGER, DANGER, DANGER, DANGER, // >40.0

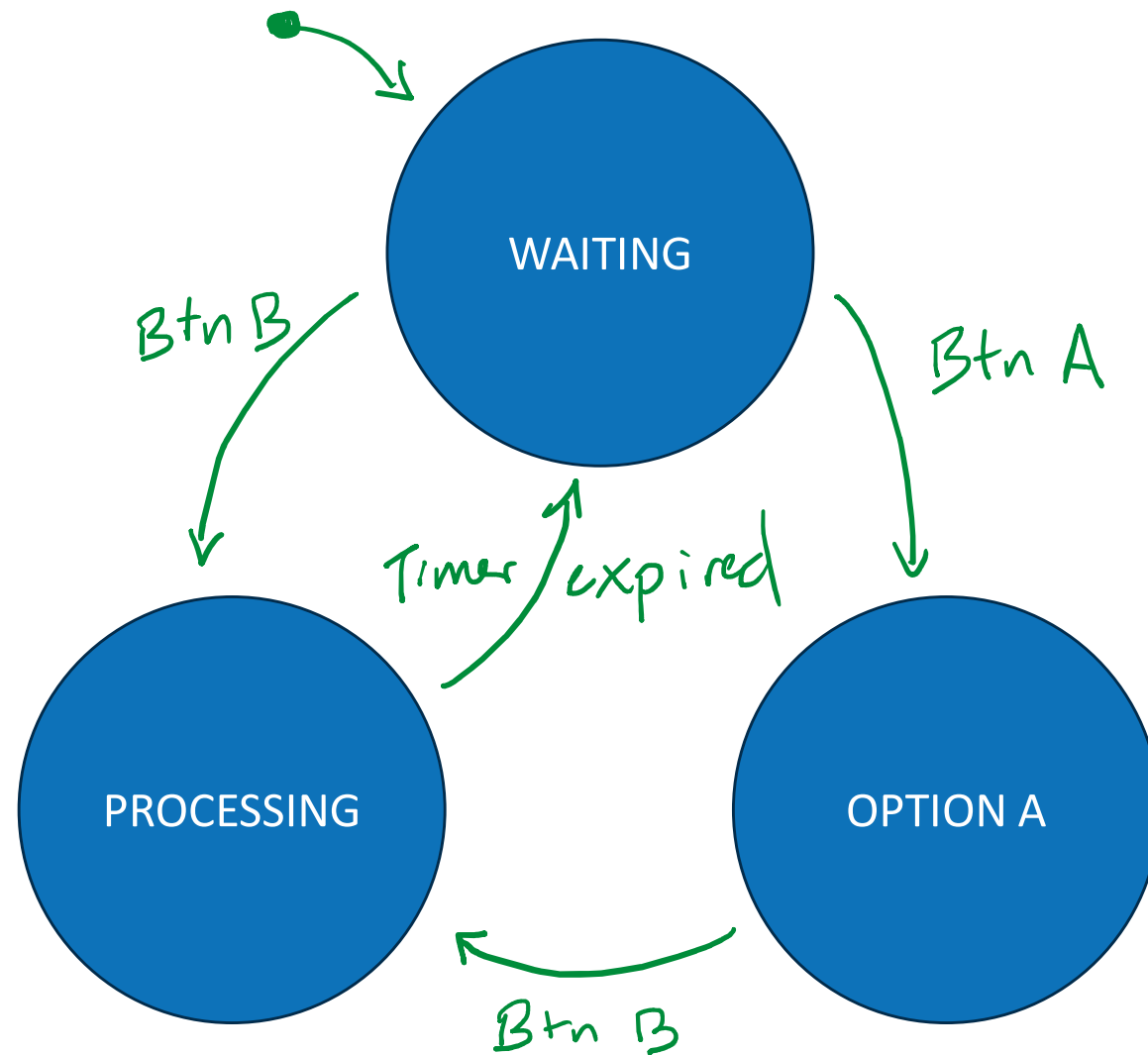
// Engine setting: Turbo
// RPMs:
//0      1000      2000      3000      4000      [5k,6k) [6k,7k) [7k,8k) 8000      9000
NORMAL,  NORMAL,  NORMAL,  NORMAL,  NORMAL,  NORMAL,  CONCERN, CONCERN, CONCERN, CONCERN, // <20.9 Temps
NORMAL,  NORMAL,  NORMAL,  NORMAL,  NORMAL,  CONCERN, CONCERN, CONCERN, WARNING, WARNING, // [20.9,25.9)
CONCERN, CONCERN, CONCERN, CONCERN, CONCERN, CONCERN, CONCERN, WARNING, WARNING, WARNING, // [25.9,29.8)
CONCERN, CONCERN, CONCERN, CONCERN, CONCERN, CONCERN, WARNING, WARNING, WARNING, WARNING, // [29.8,34.8)
CONCERN, CONCERN, CONCERN, CONCERN, CONCERN, WARNING, WARNING, WARNING, DANGER, DANGER, // [34.8,40.0)
WARNING, WARNING, WARNING, WARNING, WARNING, WARNING, WARNING, DANGER, DANGER, DANGER // >40.0
};
```

Standard					
	RPMs				
Temps	<5000	[5k,6k)	[6k,7k)	[7k,8k)	>8k
<20.9					
[20.9, 25.9)					
[25.9,29.8)					
[29.8,34.8)					
[34.8,40.0)					
>40.0					
Legend:	Normal	Concern	Warning	Danger	

Turbo					
	RPMs				
Temps	<5000	[5k,6k)	[6k,7k)	[7k,8k)	>8k
<20.9					
[20.9, 25.9)					
[25.9,29.8)					
[29.8,34.8)					
[34.8,40.0)					
>40.0					

Source: tinyurl.com/bs37kda9

Finite State Machines



Finite State Machines

```
typedef enum { WAITING, PROCESSING, OPTION_A, NUM_STATES } states_t;
typedef enum { BUTTON_A, BUTTON_B, TIMER_EXP, NUM_EVENTS } events_t;

// Function prototypes
states_t BtnA_W(void);
states_t BtnB_W(void);
...

void process_event(events_t event) {

    static states_t current_state = WAITING;

    switch( event ) {
        case BUTTON_A:
            switch( current_state ):
                case WAITING:
                    current_state = BtnA_W();
                    break;
                case PROCESSING:
                    ...
    }
```

Finite State Machines

```
typedef enum { WAITING, PROCESSING, OPTION_A, NUM_STATES } states_t;
typedef enum { BUTTON_A, BUTTON_B, TIMER_EXP, NUM_EVENTS } events_t;

// Function prototypes
states_t BtnA_W(void);
states_t BtnB_W(void);
...

void process_event(events_t event) {

    static states_t current_state = WAITING;

    static states_t (* const handlers[NUM_STATES][NUM_EVENTS])(void) = {
// Process...  BUTTON_A  BUTTON_B  TIMER_EXP  ...in...
                BtnA_W,   BtnB_W,   TmrE_W,      // WAITING state
                BtnA_P,   BtnB_P,   TmrE_P,      // PROCESSING state
                BtnA_O,   BtnB_O,   TmrE_O,      // OPTION_A state
    }
    current_state = handlers[current_state][event]();
}
```

“Jump table”



Finite State Machines

```
typedef enum { WAITING, PROCESSING, OPTION_A, NUM_STATES } states_t;
typedef enum { BUTTON_A, BUTTON_B, TIMER_EXP, NUM_EVENTS } events_t;

// Function prototypes
states_t BtnA_W(void);
states_t BtnB_W(void);
...

void process_event(events_t event) {

    static states_t current_state = WAITING;

    static states_t (* const handlers[NUM_STATES][NUM_EVENTS])(void) = {
// Process...  BUTTON_A  BUTTON_B  TIMER_EXP  ...in...
                BtnA_W,   BtnB_W,   TmrE_W,      // WAITING state
                BtnA_P,   BtnB_P,   TmrE_P,      // PROCESSING state
                BtnA_O,   BtnB_O,   TmrE_O       // OPTION_A state
    }
    current_state = handlers[current_state][event] ();
}
```



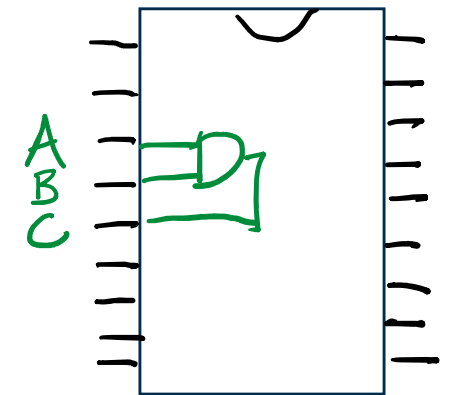
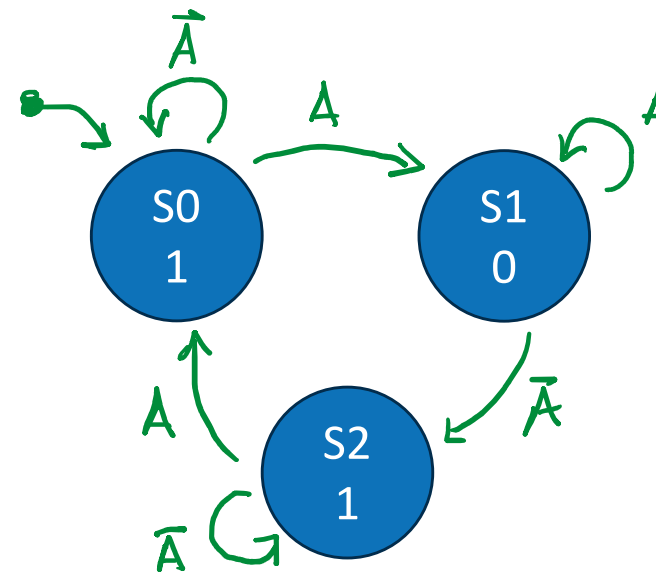
Used in appropriate circumstances, table-driven code is simpler than complicated logic, easier to modify, and more efficient.

- Steve McConnell, “Code Complete”

Hardware FSMs and Simulating Digital Logic

```
void timerInterrupt( void ) {  
    static int state = 0;  
  
    // Output in S0 S1 S2  
    int outputs[NUM_STATES] = { 1, 0, 1 };  
  
    int nextState[NUM_STATES][2*NUM_INPUTS] = {  
    // Process... !A    A    ...in...  
        0,    1,    // S0  
        2,    1,    // S1  
        2,    0    // S2  
    };  
  
    state = nextState[state][READ_PIN_A()];  
  
    if( outputs[state] ) SET_PIN_C();  
    else RESET_PIN_C();  
}
```

```
void pinChangeInterrupt( void ) {  
    int outputs[] = { 0, 0, 0, 1 };  
  
    if( outputs[(READ_PIN_A())][READ_PIN_B())] ) {  
        SET_PIN_C();  
    }  
    else {  
        RESET_PIN_C();  
    }  
}
```





Whenever I thought I needed an FPGA, I've often found I can use a software LUT instead.

- Charles Lohr, "Accomplishing the Impossible with LUTs" (paraphrased)



4

Run-time Configurability/ Simple Polymorphism

Configuration settings

```
// pin_config.h
typedef struct config_t {
    GPIO_PORT LED_PIN_PORT;
    GPIO_PIN LED_PIN_NUM;
} config_t;

const config_t system_configs[] = { { LED_PIN_PORT = GPIOA, LED_PIN_NUM = 13 },
                                     { LED_PIN_PORT = GPIOB, LED_PIN_NUM = 6  } };

// main.c
#include "pin_config.h"

void main(void) {
    while(1) {
        config_t myConfig = system_configs[READ_PIN()];

        GPIO_SET_PIN(myConfig.LED_PIN_PORT, myConfig.LED_PIN_NUM);
    }
}
```


Configuration settings

```
// pin_config.h
typedef struct config_t {
    GPIO_PORT LED_PIN_PORT;
    GPIO_PIN LED_PIN_NUM;
    int (*search)(uint8_t * arr, int key);
} config_t;

int linear(uint8_t * arr, int key);
int binary(uint8_t * arr, int key);

const config_t system_configs[] = { { LED_PIN_PORT = GPIOA, LED_PIN_NUM = 13, linear },
                                     { LED_PIN_PORT = GPIOB, LED_PIN_NUM = 6, binary } };

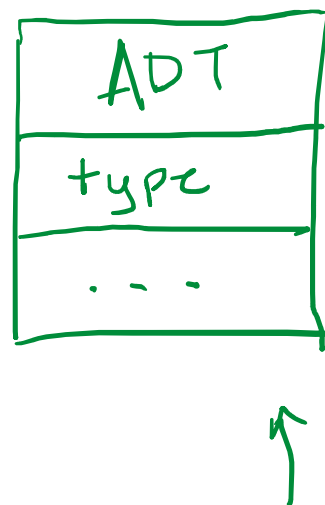
// main.c
#include "pin_config.h"

void main(void) {
    ...
    myConfig.search(myArray, 4);
}
```

Polymorphic printf

```
char * phrase[] = { "All is normal: Float val is %f\n",  
                    "Warning, Float is high: Float val is %f\n",  
                    NULL,  
                    "DANGER: Float val is %f\n"           };  
  
// Issue warning if float_val > 1.45; danger if float val is > 2.1  
//  
printf( phrase[ float_val > 2.1 ][ float_val > 1.45 ], float_val);
```

Simple Polymorphism



doStuff doMoreStuff doOtherStuff

↓ ↓ ✓

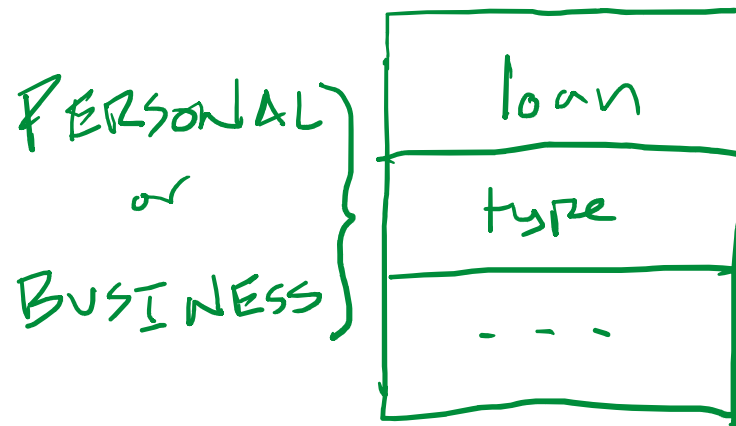
Type 0:	A()	B()	C()	...
Type 1:	D()	E()	F()	...
Type 2:	G()	H()	I()	...
...				

```
void doStuff(ADT * thisADT) {  
    interfaces[thisADT->type].doStuff(ADT);  
}
```

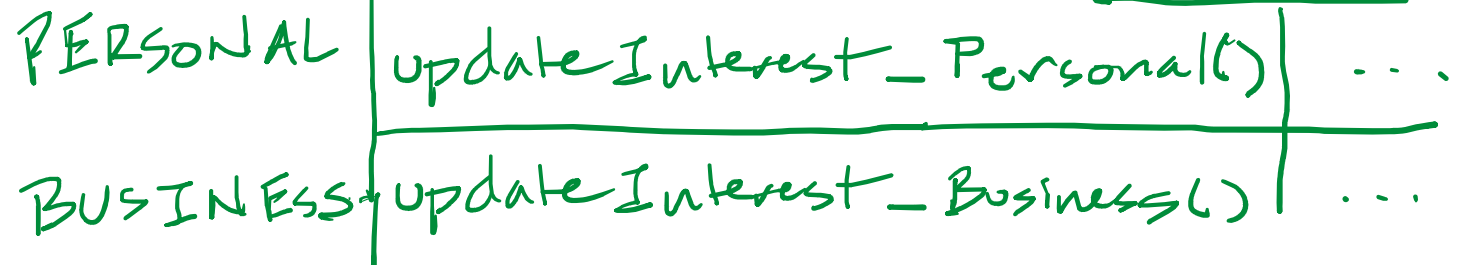
Simple Polymorphism

Ex:

```
typedef enum { PERSONAL, BUSINESS } loan_type_t;  
  
typedef struct { loan_type_t type } loan_t, *loan;
```



→



```
typedef struct { void (*updateInterest)(loan) } interfaces_t;
```

```
interfaces_t interfaces[] = {  
    [PERSONAL] = { updateInterest_Personal },  
    [BUSINESS] = { updateInterest_Business } };
```

```
void updateInterest(loan thisloan) {  
    interfaces[thisloan->type].updateInterest(thisloan);  
}
```

```
static void updateInterest_Personal(loan thisloan){ ... };  
static void updateInterest_Business(loan thisloan){ ... };
```



Capturing program knowledge as data

Previous ex. of capturing program knowledge as data

FSMs

```
static states_t (* const handlers[NUM_STATES][NUM_EVENTS])(void) = {  
// Process...  BUTTON_A  BUTTON_B  TIMER_EXP    ...in...  
               BtnA_W,   BtnB_W,   TmrE_W,      // WAITING state  
               BtnA_P,   BtnB_P,   TmrE_P,      // PROCESSING state  
               BtnA_O,   BtnB_O,   TmrE_O       // OPTION_A state  
}
```



```
static states_t (* const handlers[NUM_STATES][NUM_EVENTS])(void) = {  
// Process...  BUTTON_A  BUTTON_B  TIMER_EXP  NEW_INPUT    ...in...  
               BtnA_W,   BtnB_W,   TmrE_W,    NewIn_W,      // WAITING state  
               BtnA_P,   BtnB_P,   TmrE_P,    NewIn_P,      // PROCESSING state  
               BtnA_O,   DiffFcn,  TmrE_O,    NewIn_O       // OPTION_A state  
}
```

- New state? --> Add a row to the table.
- New input? --> Add a column to the table.
- Different action to take for a given input? --> Change the function pointer.

Previous ex. of capturing program knowledge as data

Configuration settings

```
const config_t system_configs[] = {  
    { LED_PIN_PORT = GPIOA, LED_PIN_NUM = 13, linear },  
    { LED_PIN_PORT = GPIOB, LED_PIN_NUM = 6,  binary } };
```



```
const config_t system_configs[] = {  
    { LED_PIN_PORT = GPIOA, LED_PIN_NUM = 13, linear, NewFunctionA },  
    { LED_PIN_PORT = GPIOB, LED_PIN_NUM = 6,  binary, NewFunctionB },  
    { LED_PIN_PORT = GPIOC, LED_PIN_NUM = 2,  linear, NewFunctionC } };
```

- New mode? --> Add an element to the table.
- Different LED port in mode 1? --> Modify that element in the table.
- New value or function that changes based on the mode? --> Add an element to the structs in the table.

Previous ex. of capturing program knowledge as data

Simple polymorphism

```
typedef enum { PERSONAL, BUSINESS } loan_type_t;
```

```
interfaces_t interfaces[] = {  
    [PERSONAL] = { updateInterest_Personal },  
    [BUSINESS] = { updateInterest_Business } };
```

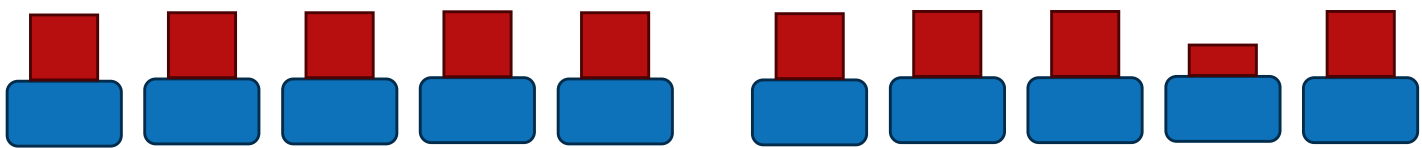
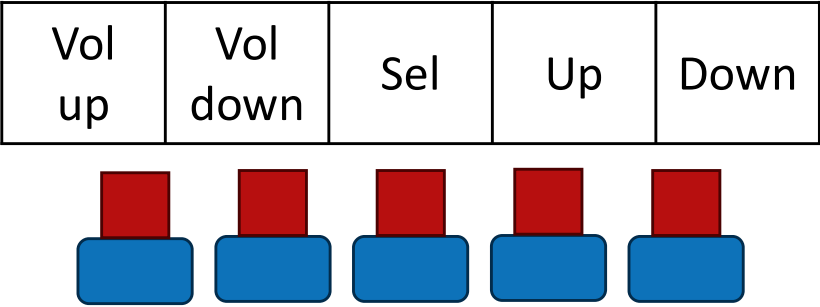
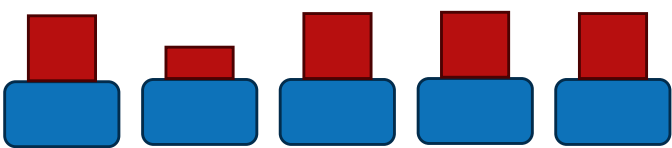


```
typedef enum { PERSONAL, BUSINESS, HOME } loan_type_t;
```

```
interfaces_t interfaces[] = {  
    [PERSONAL] = { updateInterest_Personal, makePayment_Personal },  
    [BUSINESS] = { updateInterest_Business, makePayment_Business },  
    [HOME]      = { updateInterest_Home,      makePayment_Home      } };
```

- New object type? --> Add a row to the table.
- Different behavior? --> Modify a function in the table.
- Additional behavior? --> Add an element to the struct in the table.

Button metadata



Vol up							
Vol down	Process						
Select							
Up				Process			Process
Down							

Report rate

Button metadata

```
static constexpr KeyBehaviorT m_KeyBehaviors[] = {
    { Key::KEY_VOLUME_UP,    100, "VolumeUp"  },
    { Key::KEY_VOLUME_DOWN, 100, "VolumeDown"},
    { Key::KEY_SELECT,       0,  "SelectKey" },
    { Key::KEY_UP,           250, "UpKey"     },
    { Key::KEY_DOWN,         250, "DownKey"   },
};
```

```
FUNCTION processDebounceKey
    //      New key press ..or.. Same key, held for long enough to run another time
    //
    IF( ( key != lastKey ) || ( timeSinceLastAction > m_KeyBehaviors[key].repeat_rate ) {
        takeAction = true
    }

    IF( takeAction ) /* Process key */
```

Source: tinyurl.com/2vkn4sxr

Button metadata

```
static constexpr KeyBehaviorT m_KeyBehaviors[] = {
    { Key::KEY_VOLUME_UP,    100, "VolumeUp" },
    { Key::KEY_VOLUME_DOWN, 100, "VolumeDown" },
    { Key::KEY_SELECT,       0,  "SelectKey" },
    { Key::KEY_UP,           250, "UpKey" },
    { Key::KEY_DOWN,         250, "DownKey" },
};
```

```
FUNCTION processDebounceKey
    //      New key press ..or.. Same key, held for long enough to run another time
    //
    IF( ( key != lastKey ) || ( timeSinceLastAction > m_KeyBehaviors[key].repeat_rate ) {
        takeAction = true
    }

    IF( takeAction ) /* Process key */
```

Source: tinyurl.com/2vkn4sxr

- New the repeat rates? --> Modify the values in the table.
- New key? --> Add a row to the table.
- New behavior for all keys? --> Modify business logic.
- Debug strings? --> Add string to table.

Challenge Question(s)

- What feature would you add to the “button metadata” example? How would you implement it?

```
static constexpr KeyBehaviorT m_KeyBehaviors[] = {
    { Key::KEY_VOLUME_UP,    100, "VolumeUp" },
    { Key::KEY_VOLUME_DOWN, 100, "VolumeDown" },
    { Key::KEY_SELECT,       0,  "SelectKey" },
    { Key::KEY_UP,           250, "UpKey" },
    { Key::KEY_DOWN,         250, "DownKey" },
};
```

```
FUNCTION processDebounceKey
    //      New key press ..or.. Same key, held for long enough to run another time
    //
    IF( ( key != lastKey ) || ( timeSinceLastAction > m_KeyBehaviors[key].repeat_rate ) {
        takeAction = true
    }

    IF( takeAction ) /* Process key */
```

Source: tinyurl.com/2vkn4sxr

Other metadata

Command-line interface functions

```
cliFunctions_t cliFunctions[] = {  
    // Name/      Function to      Help string  
    // arg        be called  
    //  
    { "led",      ledFunction,      "Control on board LED. Usage: [on | off]" },  
    { "i2c",      i2cFunction,      "Manual control of I2C port. Usage [read | write]" } };
```

User-provided parameters

```
// module.h  
typedef enum { RED, GREEN, BLUE } colors_t;  
void create( colors_t desiredColor, ... );
```

```
// module.c  
colorData_t colorData[] = {  
    [RED]    = { "red",    0xFF0000, /* Other color data */ },  
    [GREEN]  = { "green",  0x00FF00, /* Other color data */ },  
    [BLUE]   = { "blue",   0x0000FF, /* Other color data */ } };
```

Table-based Initialization

timerConfig_t timers[] =

TIMER_ENABLED	TIMER_DISABLED
0x40000002 (Timer 0)	0x40000022 (Timer 1)
DIV_8	N/A
1000	N/A
ISR_ENABLED	ISR_DISABLED

Timer enabled?
Base memory addr for timer registers
Clock prescaler
Overflow value
ISR enabled?

FOR EACH timer IN timers
IF ENABLED:

Set memory at (timer.baseAddr + offsetClockPrescaler) to timer.clockPrescaler
Set memory at (timer.baseAddr + offsetOverflow) to timer.overflowValue
IF ISR_ENABLED:
Set ISR enable bit
Turn on timer

Source: tinyurl.com/3f6wyn4k

Summary

Use a LUT for:

- Fast math or fast output
- Replacing long or complicated conditionals
 - Especially FSMs!
- Simulating hardware
- Adding simple polymorphic behavior/
run-time configurability
 - Configuration settings
 - Polymorphic printf
 - Simple polymorphic objects
- Grouping and accessing associated metadata
 - Button metadata
 - Help strings
 - User-provided inputs
- Creating loops/functions that operate on successive elements in a LUT
 - Table-based initialization

Challenge Question(s)

- Think of a recent project you were working on. What part(s) might have been better implemented using a LUT?

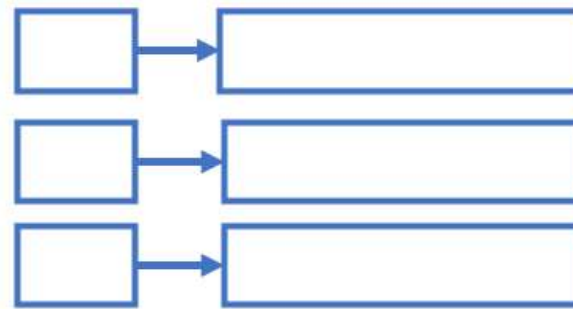
Shower Thought

- Direct ($O(1)$) access via index (natural number)
- Fixed-size collection of “items” (integers, pointers, etc)
- Can be sorted or unsorted

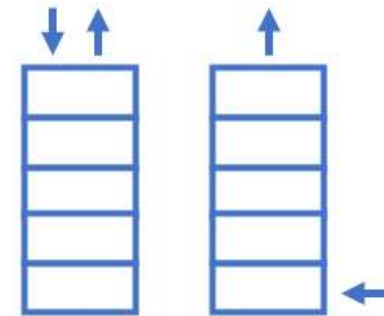
Array



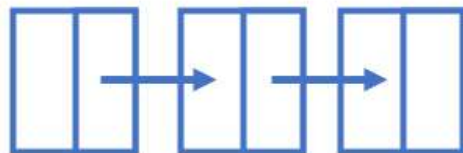
Hash Table



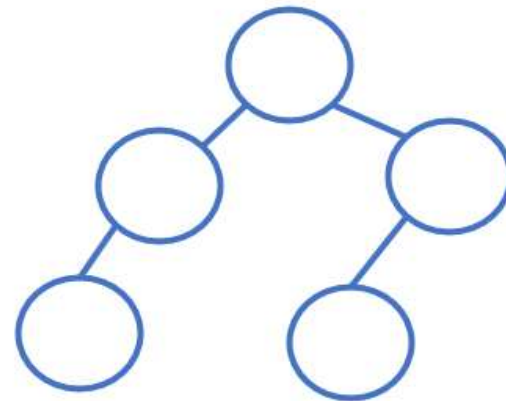
Stack & Queue



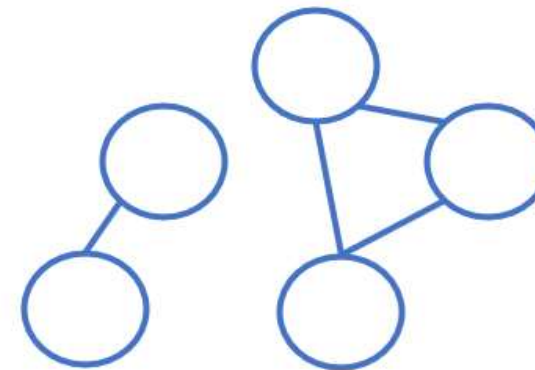
Linked List



Tree



Graph



Source: tinyurl.com/2yb4bu28

CLI Parser

>> gpio port=A pin=1 settings mode=output speed=low
Setting port A, pin 1 to OUTPUT, LOW speed

>> gpio read port=A pin=1
Value on port A, pin 1 is 0

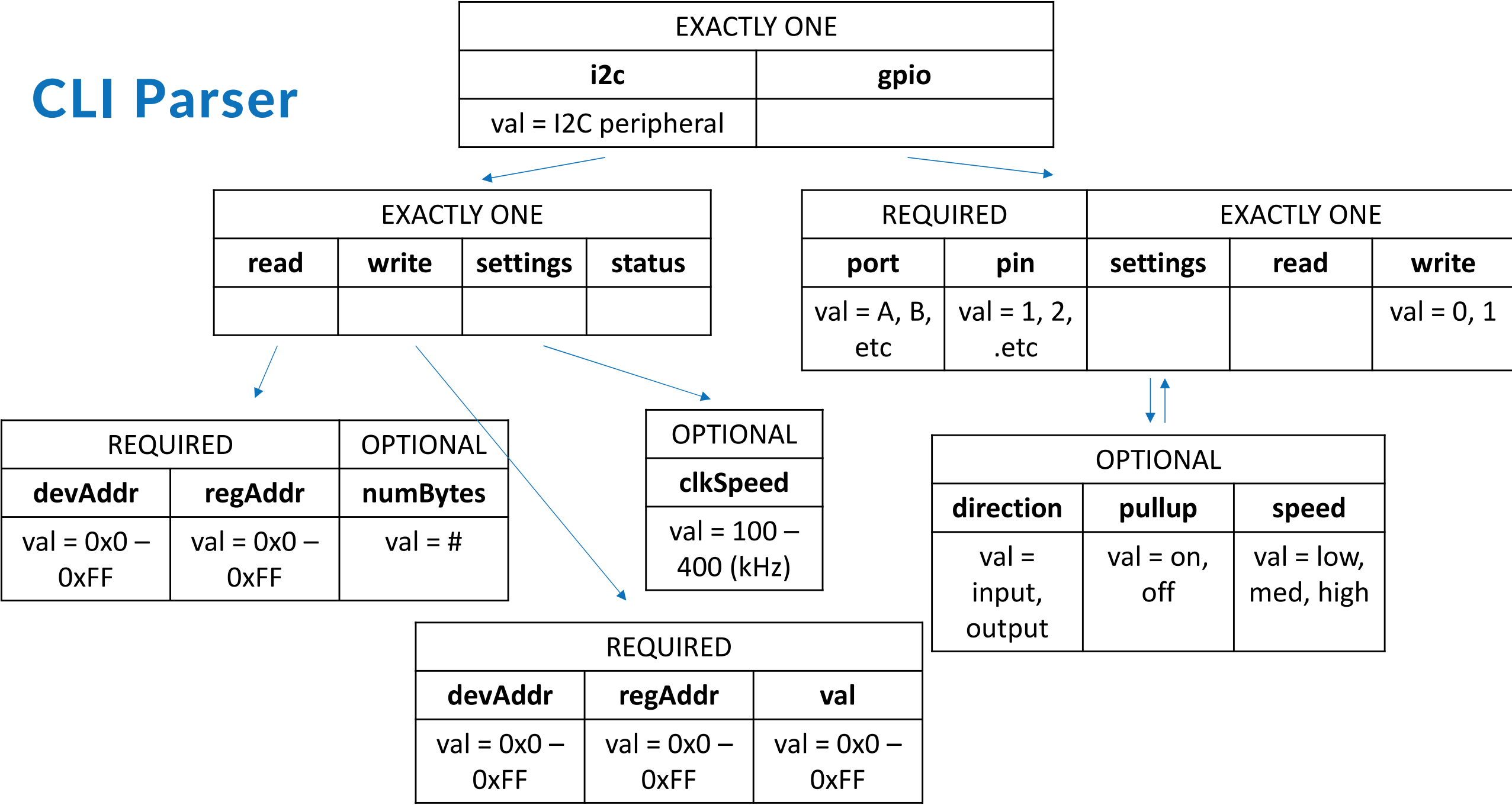
>> gpio write=0 port=A pin=1
Writing 0 to port A, pin 1

>> gpio read port=A pin=1
Value on port A, pin 1 is 1

>> i2c=1 settings clkSpeed=400
Setting clock speed on I2C1 to 400kHz

>> i2c=1 read devAddr=0x74 regAddr=0x20 bytes=2
Reading 2 byte(s) from I2C1, devAddr 0x74 at regAddr 0x20: 0x05 0x1F

CLI Parser



CLI Parser

Only 40 LOC!!
(plus 2 helper
functions, 23 LOC)

```
char * restOfInput;
char * token = strtok_r(inputString, " \n", &restOfInput);
while( token != NULL && !error ) {
    char * restOfToken;
    char * arg = strtok_r(token, "=", &restOfToken);
    char * val = strtok_r(NULL, "=", &restOfToken);
    argument_t * currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    while( ( !currentArg->argName || strcmp( arg, currentArg->argName ) != 0 ) && currentParser->prevParser != NULL ) {
        if( !currentParserSatisfied( currentParser ) ) error = true;
        currentParser = (parser_t*)currentParser->prevParser;
        currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    }
    if( currentArg->argName && strcmp( arg, currentArg->argName ) == 0 ) {
        currentArg->argGiven = true;
        switch( currentArg->argOption ) {
            case ARG_OPTIONAL:
                *(bool*)((uint8_t*)&functionParams)+currentArg->argOffset = true;
                break;
            case ARG_EXACTLY_ONE:
                *(char**)((uint8_t*)&functionParams)+currentArg->argOffset = arg;
                if( currentParser->exactlyOneReceived == false) currentParser->exactlyOneReceived = true;
                else error = true;
                break;
            case ARG_REQUIRED:
                currentArg->argGiven = true;
            default:
                break;
        }
    }
    if( val ) {
        currentArg->valGiven = true;
        if( currentArg->valOption == VAL_NONE ) error = true;
        else *(char**)((uint8_t*)&functionParams)+currentArg->valOffset = val;
    }
    else if( currentArg->valOption == VAL_REQUIRED ) error = true;
    if( currentArg->subparser ) currentParser = (parser_t*)currentArg->subparser;
}
else error = true;
token = strtok_r(NULL, " \n", &restOfInput);
}
if( !error && currentParserSatisfied(currentParser) ) actions[hash_key( *(char**)( &functionParams ) )&0xF](functionParams);
```

Source: tinyurl.com/5nfesm2p

CLI Parser

>> gpio port=A pin=1 settings mode=output speed=low
Setting port A, pin 1 to OUTPUT, LOW speed

>> gpio read port=A pin=1
Value on port A, pin 1 is 0

>> gpio write=0 port=A pin=1
Writing 0 to port A, pin 1

>> gpio read port=A pin=1
Value on port A, pin 1 is 1

>> i2c=1 settings clkSpeed=400
Setting clock speed on I2C1 to 400kHz

>> i2c=1 read devAddr=0x74 regAddr=0x20 bytes=2
Reading 2 byte(s) from I2C1, devAddr 0x74 at regAddr 0x20: 0x05 0x1F

CLI Parser

>> gpio port=A pin=1 settings mode=output speed=low
Setting port A, pin 1 to OUTPUT, LOW speed

>> gpio read port=A pin=1
Value on port A, pin 1 is 0

>> gpio write=0 port=A pin=1
Writing 0 to port A, pin 1

>> gpio read port=A pin=1
Value on port A, pin 1 is 1

>> i2c=1 settings clkSpeed=400
Setting clock speed on I2C1 to 400kHz

>> i2c=1 read devAddr=0x74 regAddr=0x20 bytes=2
Reading 2 byte(s) from I2C1, devAddr 0x74 at regAddr 0x20: 0x05 0x1F

REQUIRED
OPTIONAL
EXACTLY_ONE



Format: arg=val arg arg=val ...



REQUIRED
OPTIONAL
NONE

CLI Parser

>> gpio port=A pin=1 settings mode=output speed=low
Setting port A, pin 1 to OUTPUT, LOW speed

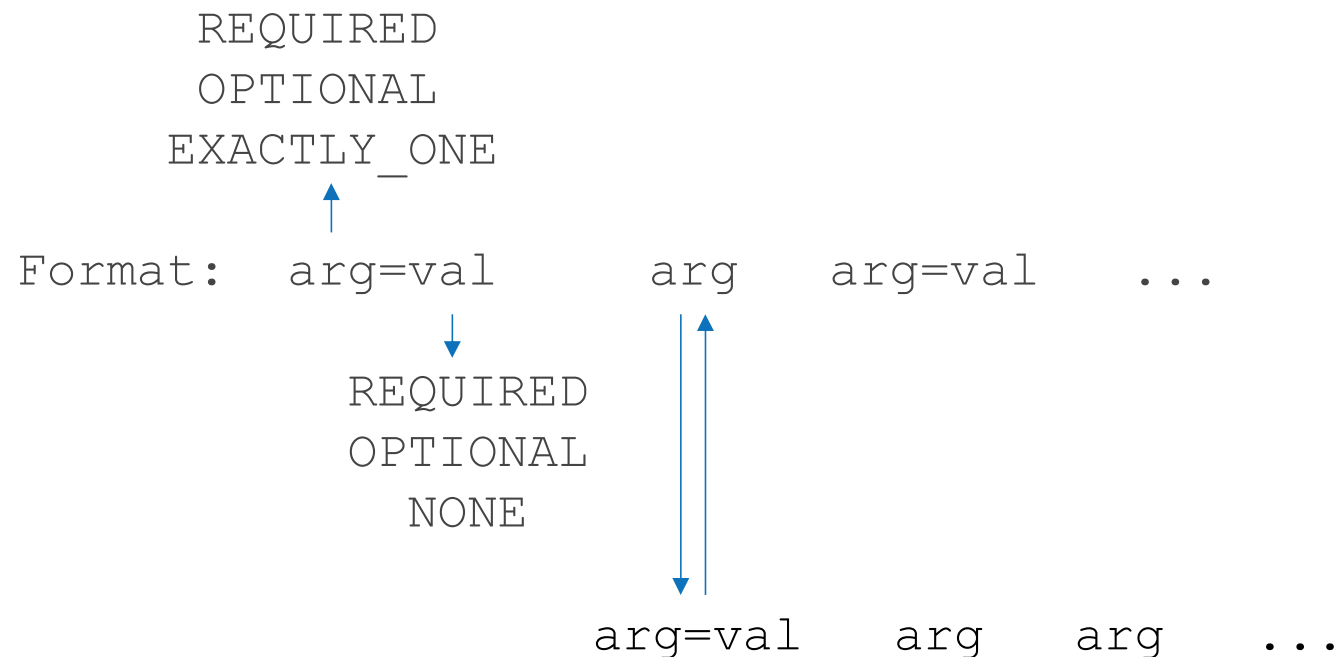
>> gpio read port=A pin=1
Value on port A, pin 1 is 0

>> gpio write=0 port=A pin=1
Writing 0 to port A, pin 1

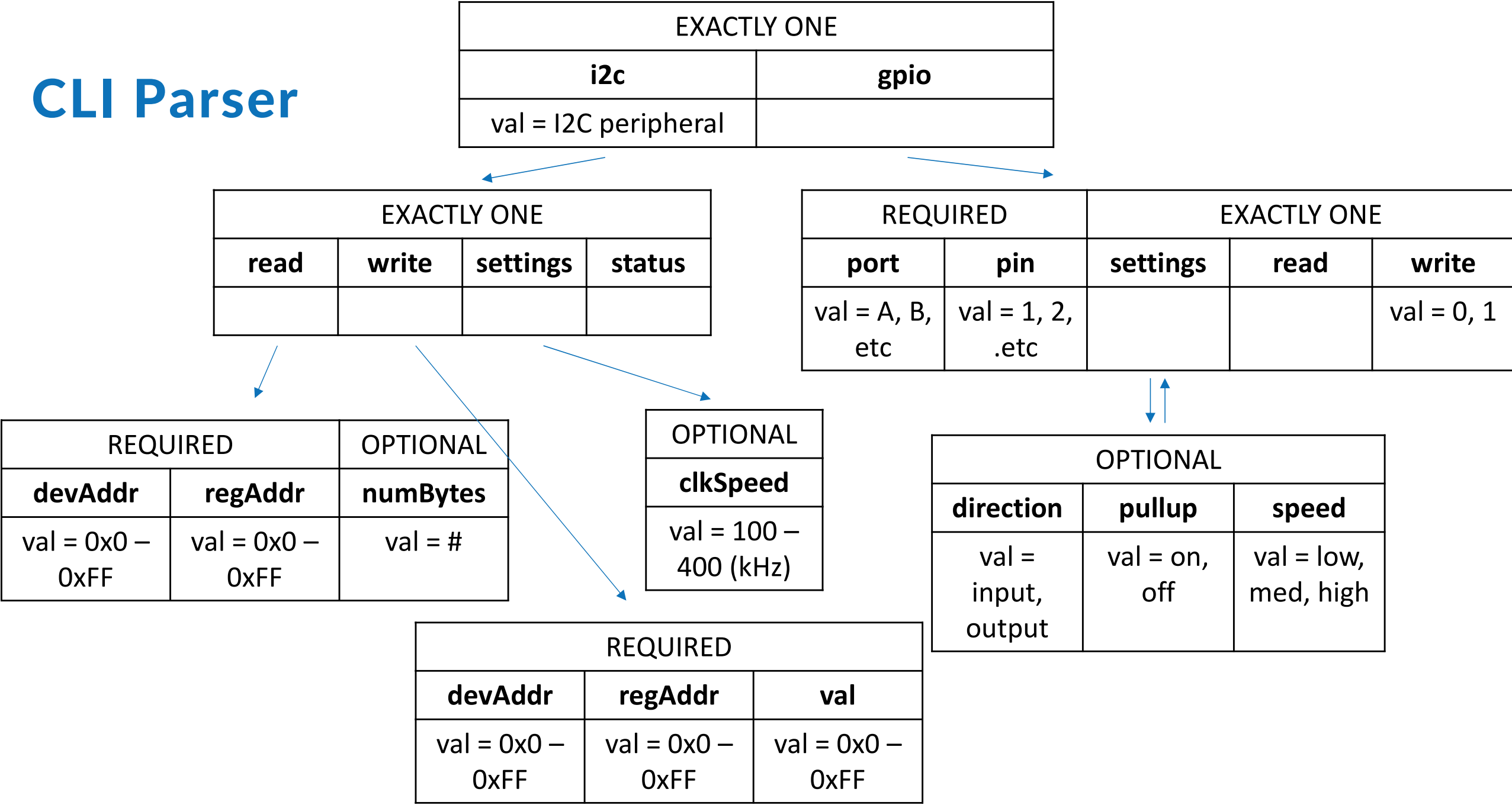
>> gpio read port=A pin=1
Value on port A, pin 1 is 1

>> i2c=1 settings clkSpeed=400
Setting clock speed on I2C1 to 400kHz

>> i2c=1 read devAddr=0x74 regAddr=0x20 bytes=2
Reading 2 byte(s) from I2C1, devAddr 0x74 at regAddr 0x20: 0x05 0x1F

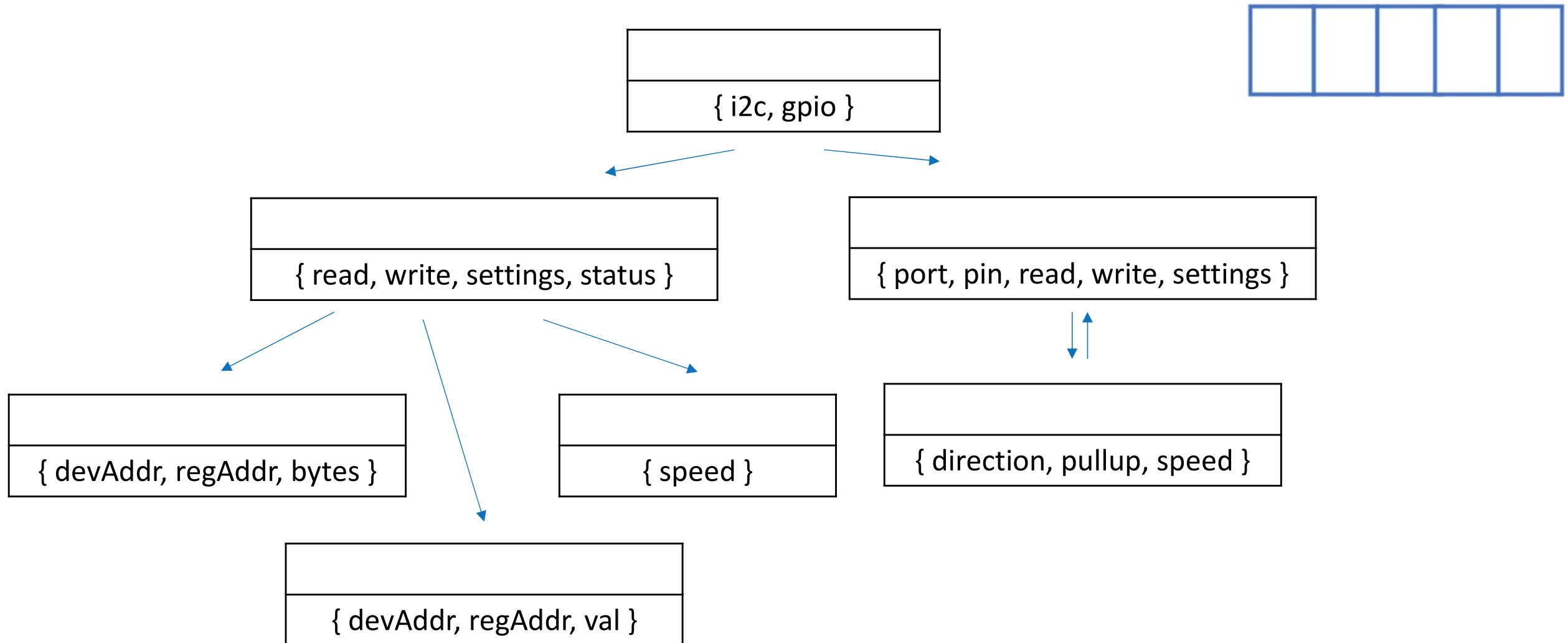


CLI Parser

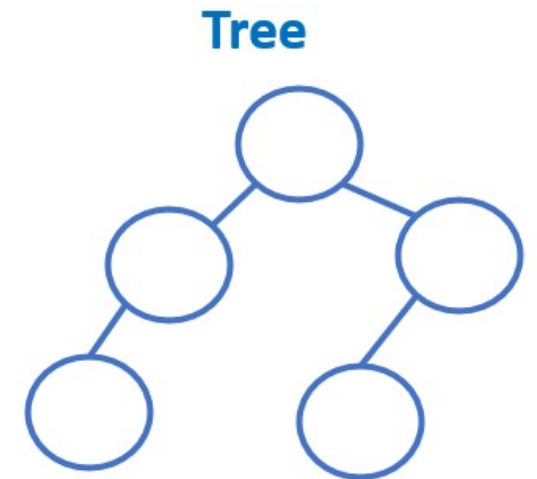
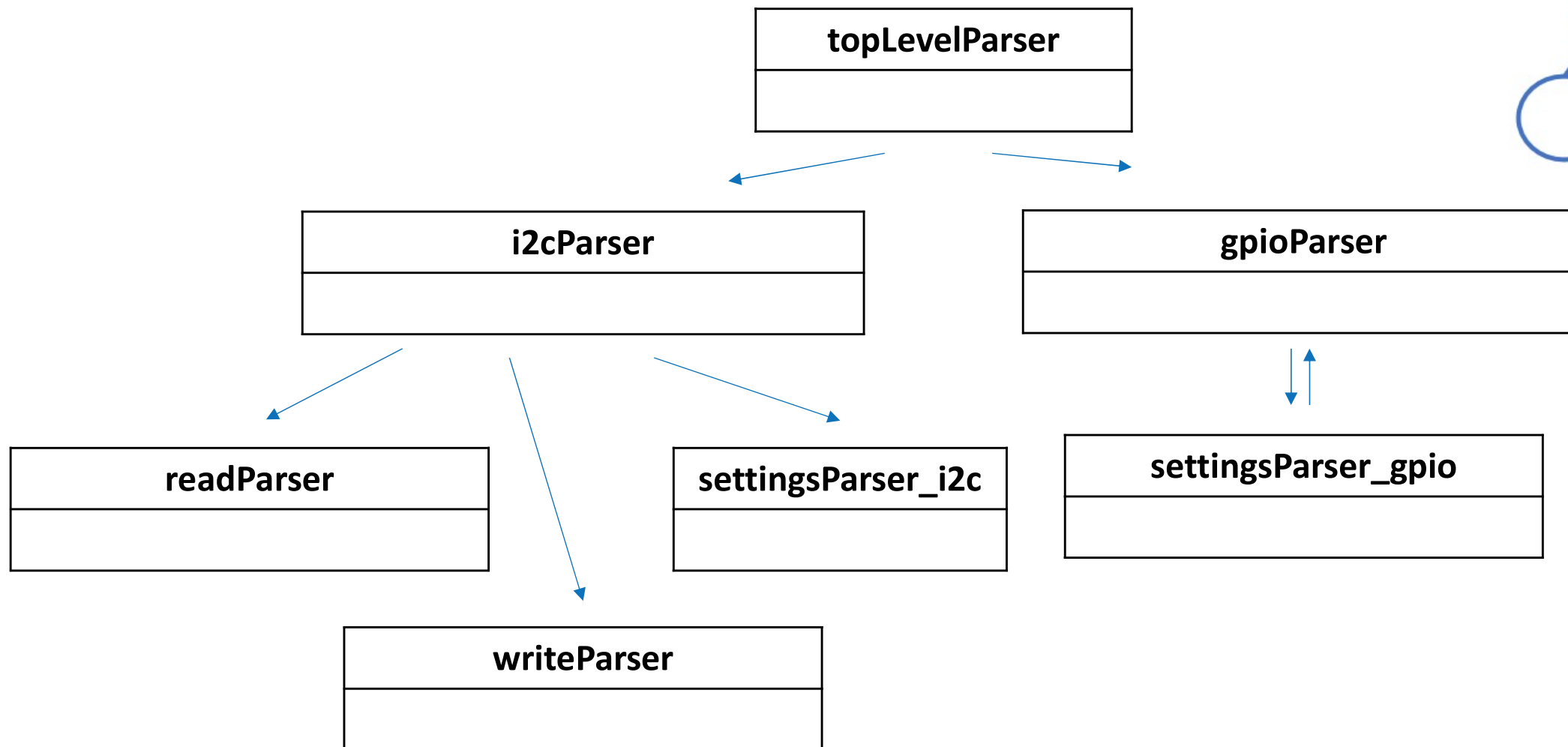


CLI Parser

Array



CLI Parser



CLI Parser

EXACTLY ONE	
i2c	gpio
val = I2C peripheral	

CLI Parser

EXACTLY ONE	
i2c	gpio
val = I2C peripheral	



"i2c"
Arg: Req'd / Opt / One
argGiven
Val: Req'd / Opt / None
valGiven

CLI Parser

EXACTLY ONE	
i2c	gpio
val = I2C peripheral	

"i2c"
Arg: Req'd / Opt / One
argOffset
argGiven
Val: Req'd / Opt / None
valOffset
valGiven

```
typedef struct{
    char * name;           // "i2c"
    char * i2cPeriph;      // 1, 2, 3
    char * subArg;         // "read" etc
    char * devAddr;
    ...
} i2c_t;
```

```
typedef union{
    gpio_t gpio;
    i2c_t i2c;
} functionParams_t;
```

```
functionParams_t functionParams = {0};
```

CLI Parser

EXACTLY ONE	
i2c	gpio
val = I2C peripheral	

"i2c"
Arg: Req'd / Opt / One
argOffset
argGiven
Val: Req'd / Opt / None
valOffset
valGiven
subparser

EXACTLY ONE			
read	write	settings	status

```
typedef struct{
    char * name;           // "i2c"
    char * i2cPeriph;      // 1, 2, 3
    char * subArg;         // "read" etc
    char * devAddr;
    ...
} i2c_t;

typedef union{
    gpio_t gpio;
    i2c_t i2c;
} functionParams_t;

functionParams_t functionParams = {0};
```

CLI Parser

EXACTLY ONE	
i2c	gpio
val = I2C peripheral	

"i2c"
Arg: Req'd / Opt / One
argOffset
argGiven
Val: Req'd / Opt / None
valOffset
valGiven
subparser

```
typedef struct{
    char * name;           // "i2c"
    char * i2cPeriph;      // 1, 2, 3
    char * subArg;         // "read" etc
    char * devAddr;
    ...
} i2c_t;
```

```
typedef union{
    gpio_t gpio;
    i2c_t i2c;
} functionParams_t;

functionParams_t functionParams = {0};
```

EXACTLY ONE			
read	write	settings	status

CLI Parser

1. Get argument
2. Find argument
3. If arg not in list:
Return to previous
parser
4. Copy argument
value to struct
5. Copy val to struct
6. Use subparser (if
present)
7. Check for errors
8. Call function

```
char * restOfInput;
char * token = strtok_r(inputString, " \n", &restOfInput);
while( token != NULL && !error ) {
    char * restOfToken;
    char * arg = strtok_r(token, "=", &restOfToken);
    char * val = strtok_r(NULL, "=", &restOfToken);
    argument_t * currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    while( ( !currentArg->argName || strcmp( arg, currentArg->argName ) != 0 ) && currentParser->prevParser != NULL ) {
        if( !currentParserSatisfied( currentParser ) ) error = true;
        currentParser = (parser_t*)currentParser->prevParser;
        currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    }
    if( currentArg->argName && strcmp( arg, currentArg->argName ) == 0 ) {
        currentArg->argGiven = true;
        switch( currentArg->argOption ) {
            case ARG_OPTIONAL:
                *(bool*)((uint8_t*)&functionParams)+currentArg->argOffset = true;
                break;
            case ARG_EXACTLY_ONE:
                *(char**)((uint8_t*)&functionParams)+currentArg->argOffset = arg;
                if( currentParser->exactlyOneReceived == false ) currentParser->exactlyOneReceived = true;
                else error = true;
                break;
            case ARG_REQUIRED:
                currentArg->argGiven = true;
            default:
                break;
        }
        if( val ) {
            currentArg->valGiven = true;
            if( currentArg->valOption == VAL_NONE ) error = true;
            else *(char**)((uint8_t*)&functionParams)+currentArg->valOffset = val;
        }
        else if( currentArg->valOption == VAL_REQUIRED ) error = true;
        if( currentArg->subparser ) currentParser = (parser_t*)currentArg->subparser;
    }
    else error = true;
    token = strtok_r(NULL, " \n", &restOfInput);
}
if( !error && currentParserSatisfied(currentParser) ) actions[hash_key( *(char**)( &functionParams ) )&0xF](functionParams);
```

Source: tinyurl.com/5nfesm2p

CLI Parser

1. Get argument
2. Find argument
3. If arg not in list:
Return to previous
parser
4. Copy argument
value to struct
5. Copy val to struct
6. Use subparser (if
present)
7. Check for errors
8. Call function

```
char * restOfInput;
char * token = strtok_r(inputString, " \n", &restOfInput);
while( token != NULL && !error ) {
    char * restOfToken;
    char * arg = strtok_r(token, "=", &restOfToken);
    char * val = strtok_r(NULL, "=", &restOfToken);
    argument_t * currentArg = &currentParser->allowableArgs[hash_key(arg) &0xF];
    while( ( !currentArg->argName || strcmp( arg, currentArg->argName ) != 0 ) && currentParser->prevParser != NULL ) {
        if( !currentParserSatisfied( currentParser ) ) error = true;
        currentParser = (parser_t*)currentParser->prevParser;
        currentArg = &currentParser->allowableArgs[hash_key(arg) &0xF];
    }
    if( currentArg->argName && strcmp( arg, currentArg->argName ) == 0 ) {
        currentArg->argGiven = true;
        switch( currentArg->argOption ) {
            case ARG_OPTIONAL:
                *(bool*)((uint8_t*)&functionParams)+currentArg->argOffset = true;
                break;
            case ARG_EXACTLY_ONE:
                *(char*)((uint8_t*)&functionParams)+currentArg->argOffset = arg;
                if( currentParser->exactlyOneReceived == false ) currentParser->exactlyOneReceived = true;
                else error = true;
                break;
            case ARG_REQUIRED:
                currentArg->argGiven = true;
            default:
                break;
        }
        if( val ) {
            currentArg->valGiven = true;
            if( currentArg->valOption == VAL_NONE ) error = true;
            else *(char*)((uint8_t*)&functionParams)+currentArg->valOffset = val;
        }
        else if( currentArg->valOption == VAL_REQUIRED ) error = true;
        if( currentArg->subparser ) currentParser = (parser_t*)currentArg->subparser;
    }
    else error = true;
    token = strtok_r(NULL, " \n", &restOfInput);
}
if( !error && currentParserSatisfied(currentParser) ) actions[hash_key( *(char*)((uint8_t*)&functionParams) ) &0xF](functionParams);
```



Source: tinyurl.com/5nfesm2p

CLI Parser

1. Get argument
2. Find argument
3. If arg not in list:
Return to
previous parser
4. Copy argument
value to struct
5. Copy val to struct
6. Use subparser (if
present)
7. Check for errors
8. Call function

```
char * restOfInput;
char * token = strtok_r(inputString, " \n", &restOfInput);
while( token != NULL && !error ) {
    char * restOfToken;
    char * arg = strtok_r(token, "=", &restOfToken);
    char * val = strtok_r(NULL, "=", &restOfToken);
    argument_t * currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    while( ( !currentArg->argName || strcmp( arg, currentArg->argName ) != 0 ) && currentParser->prevParser != NULL ) {
        if( !currentParserSatisfied( currentParser ) ) error = true;
        currentParser = (parser_t*)currentParser->prevParser;
        currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    }
    if( currentArg->argName && strcmp( arg, currentArg->argName ) == 0 ) {
        currentArg->argGiven = true;
        switch( currentArg->argOption ) {
            case ARG_OPTIONAL:
                *(bool*)((uint8_t*)&functionParams)+currentArg->argOffset = true;
                break;
            case ARG_EXACTLY_ONE:
                *(char*)((uint8_t*)&functionParams)+currentArg->argOffset = arg;
                if( currentParser->exactlyOneReceived == false ) currentParser->exactlyOneReceived = true;
                else error = true;
                break;
            case ARG_REQUIRED:
                currentArg->argGiven = true;
            default:
                break;
        }
        if( val ) {
            currentArg->valGiven = true;
            if( currentArg->valOption == VAL_NONE ) error = true;
            else *(char*)((uint8_t*)&functionParams)+currentArg->valOffset = val;
        }
        else if( currentArg->valOption == VAL_REQUIRED ) error = true;
        if( currentArg->subparser ) currentParser = (parser_t*)currentArg->subparser;
    }
    else error = true;
    token = strtok_r(NULL, " \n", &restOfInput);
}
if( !error && currentParserSatisfied(currentParser) ) actions[hash_key( *(char*)((uint8_t*)&functionParams) )&0xF](functionParams);
```



Source: tinyurl.com/5nfesm2p

CLI Parser

1. Get argument
2. Find argument
3. If arg not in list:
Return to previous
parser
4. **Copy argument
value to struct**
5. Copy val to struct
6. Use subparser (if
present)
7. Check for errors
8. Call function

```
char * restOfInput;
char * token = strtok_r(inputString, " \n", &restOfInput);
while( token != NULL && !error ) {
    char * restOfToken;
    char * arg = strtok_r(token, "=", &restOfToken);
    char * val = strtok_r(NULL, "=", &restOfToken);
    argument_t * currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    while( ( !currentArg->argName || strcmp( arg, currentArg->argName ) != 0 ) && currentParser->prevParser != NULL ) {
        if( !currentParserSatisfied( currentParser ) ) error = true;
        currentParser = (parser_t*)currentParser->prevParser;
        currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    }
    if( currentArg->argName && strcmp( arg, currentArg->argName ) == 0 ) {
        currentArg->argGiven = true;
        switch( currentArg->argOption ) {
            case ARG_OPTIONAL:
                *(bool*)((uint8_t*)&functionParams+currentArg->argOffset) = true;
                break;
            case ARG_EXACTLY_ONE:
                *(char*)((uint8_t*)&functionParams+currentArg->argOffset) = arg;
                if( currentParser->exactlyOneReceived == false ) currentParser->exactlyOneReceived = true;
                else error = true;
                break;
            case ARG_REQUIRED:
                currentArg->argGiven = true;
            default:
                break;
        }
        if( val ) {
            currentArg->valGiven = true;
            if( currentArg->valOption == VAL_NONE ) error = true;
            else *(char*)((uint8_t*)&functionParams+currentArg->valOffset) = val;
        }
        else if( currentArg->valOption == VAL_REQUIRED ) error = true;
        if( currentArg->subparser ) currentParser = (parser_t*)currentArg->subparser;
    }
    else error = true;
    token = strtok_r(NULL, " \n", &restOfInput);
}
if( !error && currentParserSatisfied(currentParser) ) actions[hash_key( *(char*)((uint8_t*)&functionParams) )&0xF](functionParams);
```

Source: tinyurl.com/5nfesm2p

CLI Parser

1. Get argument
2. Find argument
3. If arg not in list:
Return to previous
parser
4. Copy argument
value to struct
5. **Copy val to struct**
6. Use subparser (if
present)
7. Check for errors
8. Call function

```
char * restOfInput;
char * token = strtok_r(inputString, " \n", &restOfInput);
while( token != NULL && !error ) {
    char * restOfToken;
    char * arg = strtok_r(token, "=", &restOfToken);
    char * val = strtok_r(NULL, "=", &restOfToken);
    argument_t * currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    while( ( !currentArg->argName || strcmp( arg, currentArg->argName ) != 0 ) && currentParser->prevParser != NULL ) {
        if( !currentParserSatisfied( currentParser ) ) error = true;
        currentParser = (parser_t*)currentParser->prevParser;
        currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    }
    if( currentArg->argName && strcmp( arg, currentArg->argName ) == 0 ) {
        currentArg->argGiven = true;
        switch( currentArg->argOption ) {
            case ARG_OPTIONAL:
                *(bool*)((uint8_t*)&functionParams)+currentArg->argOffset = true;
                break;
            case ARG_EXACTLY_ONE:
                *(char**)((uint8_t*)&functionParams)+currentArg->argOffset = arg;
                if( currentParser->exactlyOneReceived == false ) currentParser->exactlyOneReceived = true;
                else error = true;
                break;
            case ARG_REQUIRED:
                currentArg->argGiven = true;
            default:
                break;
        }
    }
    if( val ) {
        currentArg->valGiven = true;
        if( currentArg->valOption == VAL_NONE ) error = true;
        else *(char**)((uint8_t*)&functionParams)+currentArg->valOffset = val;
    }
    else if( currentArg->valOption == VAL_REQUIRED ) error = true;
    if( currentArg->subparser ) currentParser = (parser_t*)currentArg->subparser;
}
else error = true;
token = strtok_r(NULL, " \n", &restOfInput);
}
if( !error && currentParserSatisfied(currentParser) ) actions[hash_key( *(char**)( &functionParams ) &0xF )(functionParams);
```

Source: tinyurl.com/5nfesm2p

CLI Parser

1. Get argument
2. Find argument
3. If arg not in list:
Return to previous
parser
4. Copy argument
value to struct
5. Copy val to struct
6. **Use subparser (if
present)**
7. Check for errors
8. Call function

```
char * restOfInput;
char * token = strtok_r(inputString, " \n", &restOfInput);
while( token != NULL && !error ) {
    char * restOfToken;
    char * arg = strtok_r(token, "=", &restOfToken);
    char * val = strtok_r(NULL, "=", &restOfToken);
    argument_t * currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    while( ( !currentArg->argName || strcmp( arg, currentArg->argName ) != 0 ) && currentParser->prevParser != NULL ) {
        if( !currentParserSatisfied( currentParser ) ) error = true;
        currentParser = (parser_t*)currentParser->prevParser;
        currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    }
    if( currentArg->argName && strcmp( arg, currentArg->argName ) == 0 ) {
        currentArg->argGiven = true;
        switch( currentArg->argOption ) {
            case ARG_OPTIONAL:
                *(bool*)((uint8_t*)&functionParams)+currentArg->argOffset = true;
                break;
            case ARG_EXACTLY_ONE:
                *(char**)((uint8_t*)&functionParams)+currentArg->argOffset = arg;
                if( currentParser->exactlyOneReceived == false ) currentParser->exactlyOneReceived = true;
                else error = true;
                break;
            case ARG_REQUIRED:
                currentArg->argGiven = true;
            default:
                break;
        }
        if( val ) {
            currentArg->valGiven = true;
            if( currentArg->valOption == VAL_NONE ) error = true;
            else *(char**)((uint8_t*)&functionParams)+currentArg->valOffset = val;
        }
        else if( currentArg->valOption == VAL_REQUIRED ) error = true;
        if( currentArg->subparser ) currentParser = (parser_t*)currentArg->subparser;
    }
    else error = true;
    token = strtok_r(NULL, " \n", &restOfInput);
}
if( !error && currentParserSatisfied(currentParser) ) actions[hash_key( *(char**)( &functionParams ) &0xF )(functionParams);
```

Source: tinyurl.com/5nfesm2p



CLI Parser

1. Get argument
2. Find argument
3. If arg not in list:
Return to previous
parser
4. Copy argument
value to struct
5. Copy val to struct
6. Use subparser (if
present)
7. **Check for errors**
8. Call function

```
char * restOfInput;
char * token = strtok_r(inputString, " \n", &restOfInput);
while( token != NULL && !error ) {
    char * restOfToken;
    char * arg = strtok_r(token, "=", &restOfToken);
    char * val = strtok_r(NULL, "=", &restOfToken);
    argument_t * currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    while( ( !currentArg->argName || strcmp( arg, currentArg->argName ) != 0 ) && currentParser->prevParser != NULL ) {
        if( !currentParserSatisfied( currentParser ) ) error = true;
        currentParser = (parser_t*)currentParser->prevParser;
        currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    }
    if( currentArg->argName && strcmp( arg, currentArg->argName ) == 0 ) {
        currentArg->argGiven = true;
        switch( currentArg->argOption ) {
            case ARG_OPTIONAL:
                *(bool*)((uint8_t*)&functionParams)+currentArg->argOffset = true;
                break;
            case ARG_EXACTLY_ONE:
                *(char**)((uint8_t*)&functionParams)+currentArg->argOffset = arg;
                if( currentParser->exactlyOneReceived == false ) currentParser->exactlyOneReceived = true;
                else error = true;
                break;
            case ARG_REQUIRED:
                currentArg->argGiven = true;
            default:
                break;
        }
    }
    if( val ) {
        currentArg->valGiven = true;
        if( currentArg->valOption == VAL_NONE ) error = true;
        else *(char**)((uint8_t*)&functionParams)+currentArg->valOffset = val;
    }
    else if( currentArg->valOption == VAL_REQUIRED ) error = true;
    if( currentArg->subparser ) currentParser = (parser_t*)currentArg->subparser;
}
else error = true;
token = strtok_r(NULL, " \n", &restOfInput);
if( !error && currentParserSatisfied(currentParser) ) actions[hash_key( *(char**)( &functionParams ) &0xF )(functionParams);
```

Source: tinyurl.com/5nfesm2p

CLI Parser

1. Get argument
2. Find argument
3. If arg not in list:
Return to previous
parser
4. Copy argument
value to struct
5. Copy val to struct
6. Use subparser (if
present)
7. Check for errors
8. **Call function**

```
char * restOfInput;
char * token = strtok_r(inputString, " \n", &restOfInput);
while( token != NULL && !error ) {
    char * restOfToken;
    char * arg = strtok_r(token, "=", &restOfToken);
    char * val = strtok_r(NULL, "=", &restOfToken);
    argument_t * currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    while( ( !currentArg->argName || strcmp( arg, currentArg->argName ) != 0 ) && currentParser->prevParser != NULL ) {
        if( !currentParserSatisfied( currentParser ) ) error = true;
        currentParser = (parser_t*)currentParser->prevParser;
        currentArg = &currentParser->allowableArgs[hash_key(arg)&0xF];
    }
    if( currentArg->argName && strcmp( arg, currentArg->argName ) == 0 ) {
        currentArg->argGiven = true;
        switch( currentArg->argOption ) {
            case ARG_OPTIONAL:
                *(bool*)((uint8_t*)&functionParams)+currentArg->argOffset = true;
                break;
            case ARG_EXACTLY_ONE:
                *(char*)((uint8_t*)&functionParams)+currentArg->argOffset = arg;
                if( currentParser->exactlyOneReceived == false ) currentParser->exactlyOneReceived = true;
                else error = true;
                break;
            case ARG_REQUIRED:
                currentArg->argGiven = true;
            default:
                break;
        }
    }
    if( val ) {
        currentArg->valGiven = true;
        if( currentArg->valOption == VAL_NONE ) error = true;
        else *(char*)((uint8_t*)&functionParams)+currentArg->valOffset = val;
    }
    else if( currentArg->valOption == VAL_REQUIRED ) error = true;
    if( currentArg->subparser ) currentParser = (parser_t*)currentArg->subparser;
}
else error = true;
token = strtok_r(NULL, " \n", &restOfInput);
}
if( !error && currentParserSatisfied(currentParser) ) actions[hash_key( *(char*)((uint8_t*)&functionParams) )&0xF](functionParams);
```



Source: tinyurl.com/5nfesm2p

CLI Parser

```
main.c x .replit + ...
main.c > f main Format
276 //printf("Output write\n");
277 if( error || !
currentParserSatisfied(currentParser) )
printf("Error parsing input string!\n");
278 else actions[hash_key( *(char**)
(&functionParams) )&0xF](functionParams);
279
280 printf("\n>> ");
281 memset(inputString, 0, 64);
282 int c;
283 while ((c = getchar()) != '\n' && c !=
= EOF);
284 scanf("%[^\\n]", inputString);
285 }
286 }
287
288 void gpio(functionParams_t params)
289 {
290 typedef struct
291 {
292 enum { INPUT, OUTPUT } mode;
293 enum { OFF, ON } pullup;
```

```
>_ Console x Shell +
Run
>> gpio port=A pin=1 settings
Direction of port A, pin 1 is INPUT
Pullup on port A, pin 1 is OFF
Speed of port A, pin 1 is LOW

>> gpio port=A pin=1 read
Value on port A, pin 1 is 1

>> gpio port=A pin=2 settings direction=OUTPUT
Setting direction of port A, pin 2 to OUTPUT

>> gpio port=A pin=2 read
Value on port A, pin 2 is 0

>> gpio port=A write=1 pin=2
Writing 1 to port A, pin 2

>> gpio read port=A pin=2
Value on port A, pin 2 is 1

>> i2c=1 status
Status on I2C1 is 6
1: I2C1 IS enabled
1: I2C1 ISR IS enabled
0: There is NOT a write collision error on I2C1

>> i2c=1 settings speed=400
Setting clock speed on I2C1 to 400 kHz
```

Source: tinyurl.com/5nfesm2p



The fact that a design uses inheritance and polymorphism doesn't make it a good design. The key design insight in this case is neither object orientation nor functional orientation—it's the use of a well thought out lookup table.

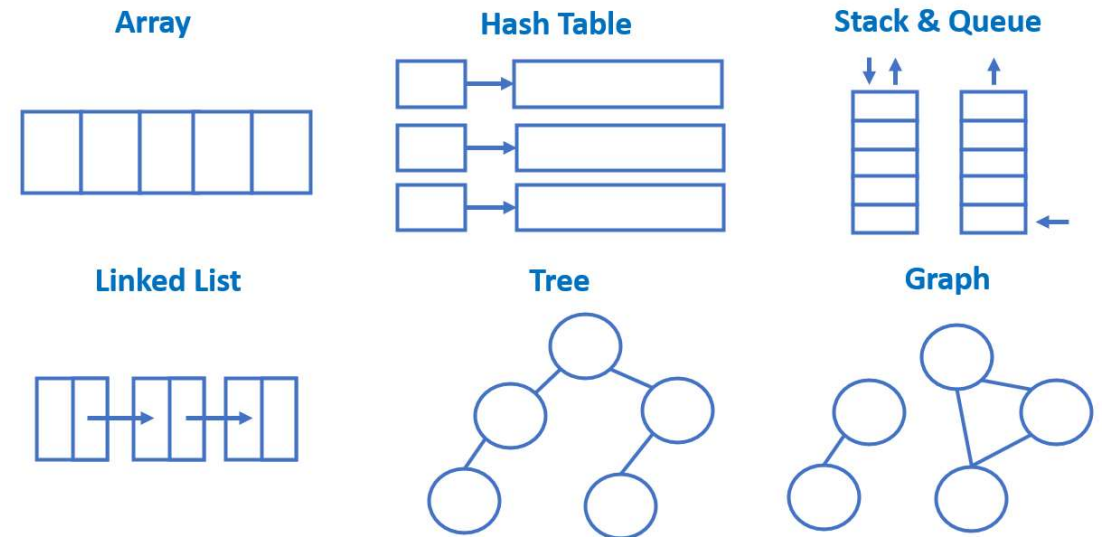
- Steve McConnell, "Code Complete"

Summary & Shower Thought

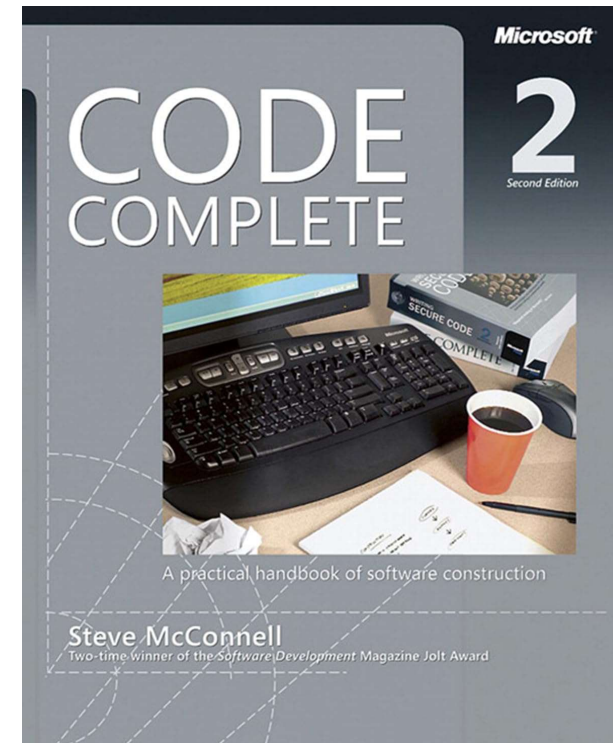
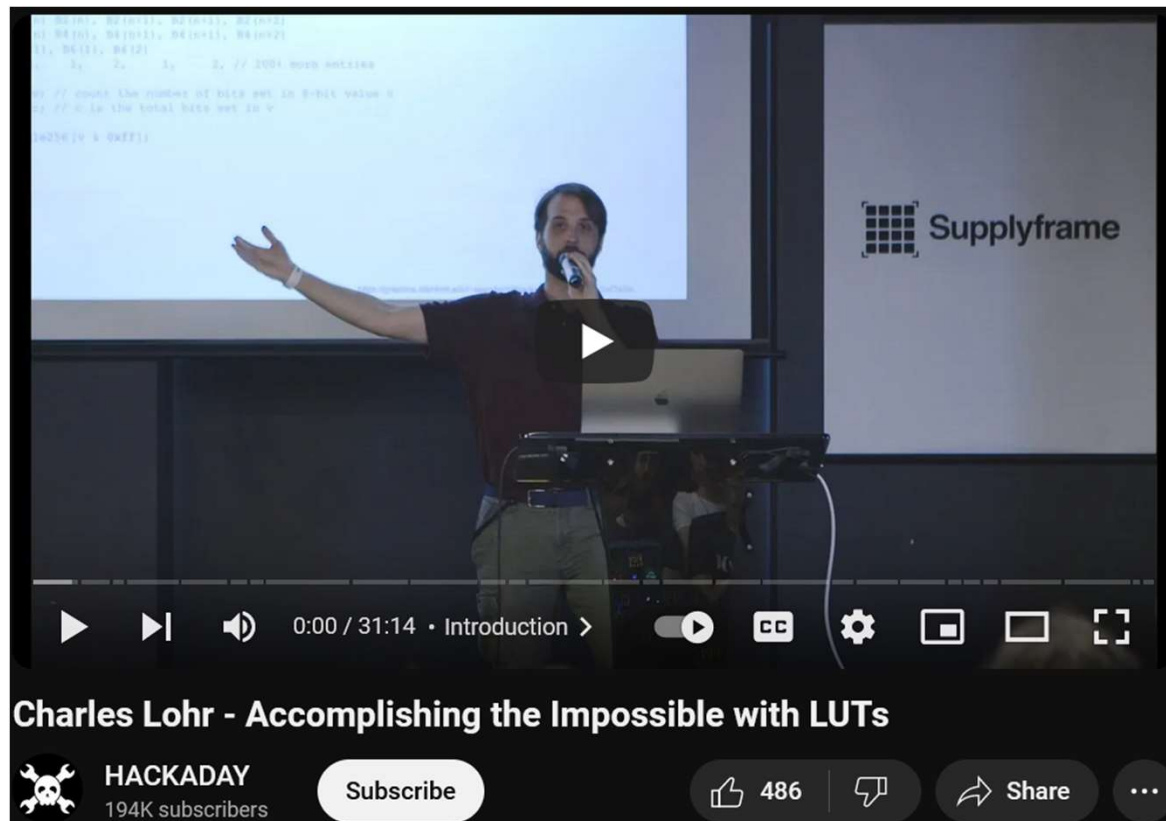
Use a LUT for:

- Fast math or fast output
- Replacing long or complicated conditionals
 - Especially FSMs!
- Simulating hardware
- Adding simple polymorphic behavior/run-time configurability
 - Configuration settings
 - Polymorphic printf
 - Simple polymorphic objects
- Grouping and accessing associated metadata
 - Button metadata
 - Help strings
 - User-provided inputs
- Creating loops/functions that operate on successive elements in a LUT
 - Table-based initialization

What software could you write if you were only the pick the right data structure?



Going Further



Chapter 18, “Table-Driven Methods”

<https://www.github.com/nathancharlesjones/Look-up-Table-Examples>

THANK YOU

Embedded
Online
Conference

w w w . e m b e d d e d o n l i n e c o n f e r e n c e . c o m